

The Thinking Machine, Part 2: Artha ani Uttara

August 28, 2026

THE THINKING MACHINE

BOOK 2, PART 2 OF 3: अर्थ आणि उत्तर

BOOK 2 चा नकाशा (3 parts)

PART 1	अंदाजाचं यंत्र [तुम्ही इथे आहात]	झाला
PART 2	अर्थ आणि उत्तर	अर्थाचे numbers, attention, transformer; file उत्तर कशी देते
PART 3	चुका, हात आणि पैसा	hallucination, तपासणी; agents, Claude Code; AI चा पैसा-नकाशा; अंतिम परीक्षा

आधी Part 1 ची परीक्षा

पाच प्रश्न. आधी स्वतः उत्तर द्या, मग खाली बघा. अडलात तर सोबतचा chapter (क्रमांक Book 2 चे).

1. उत्तर “टपकत” का येतं, आणि लांब उत्तर महाग का? (1.1)
2. "strawberry मध्ये r किती" यंत्र का चुकतं? (1.6)
3. यंत्र शिकताना आत नक्की काय बदलतं? (2.3, 2.4)
4. Base model आणि ChatGPT मध्ये फरक काय? (2.9)
5. "नवं model आलं" या वाक्याचा नेमका अर्थ? (2.10)

उत्तर: 1. उत्तर एका वेळेस एकाच token ने जन्मतं; प्रत्येक token = एक पूर्ण चक्र = GPU-वेळ; म्हणून output, input च्या 5-6 पट महाग. 2. यंत्र अक्षरं बघतच नाही; tokens (अक्षर-गट्टे) बघतं: चष्याखालचं त्याला अंधारं. 3. वजनं (weights): कोट्यवधी tuning-काटे केसा-केसाने सरकतात: अंदाज-चूक-उतार-पाऊल या चक्राने; नोंदी साठत नाहीत, दाब मुरतो. 4. Base = फक्त पुढचा-token चा आरसा: प्रवाही पण निरुपयोगी; ChatGPT = त्यावर माणसाची शिकवणी (सूचना-नमुने + RLHF ची पसंती-घोटार्ई). 5. Company ने नवी गोठवलेली वजनांची file प्रकाशित केली: नवी शाळा-धाव, नवं नाव.

या part मध्ये काय आहे

Part 1 ने एक प्रश्न मुद्दाम शिल्लक ठेवला होता: पुढचा token ओळखताना यंत्र पूर्ण रांगेकडे **बघतं कसं?** या part मध्ये त्या प्रश्नाचं पूर्ण उत्तर आहे: आणि ते उत्तर म्हणजे गेल्या दशकातली सगळ्यात मोठी तांत्रिक कल्पना.

Section 3 (11 chapters): अर्थ numbers मध्ये कसा मावतो (king आणि queen “जवळ” का राहतात), एका शब्दाचे अनेक अर्थ यंत्र कसं सोडवतं, **attention** म्हणजे नक्की काय, आणि **transformer**: GPT/Claude/Gemini या सगळ्यांची एकच मूळ रचना. शेवटी: मोठं केल्याने हुशार का होत गेलं, आणि photo-आवाज-video ही त्याच यंत्रात कसे शिरले.

Section 4 (5 chapters): गोठलेली file **उत्तर** कशी देते: एका request च्या आत काय घडतं, पहिला शब्द हळू का, तोच प्रश्न- वेगळं उत्तर का, आणि हे यंत्र “विसरतं” का.

हे तुमच्या business decision मध्ये कुठे येईल: “Context window”, “attention”, “multimodal”, “time to first token”: ही आजची खरेदी-विक्रीची भाषा आहे: pricing-पानांवर, करारांत, बातम्यांत. या part नंतर ती भाषा तुम्ही **बोलू** शकाल: आणि सगळ्यात महत्वाचं: तुमच्या AI-product चा वेग, खर्च आणि “विसरण्याच्या” तक्रारी: तिन्हींची मुळं इथेच आहेत.

SECTION 3: यंत्र अर्थ कसा हाताळतं

अकरा chapters. Token च्या number पासून transformer पर्यंत: या पुस्तकाचा सगळ्यात खोल भाग. DEPTH chapters इथे चार आहेत (3.5, 3.6, 3.7, 3.11): पहिल्या फेरीत वगळलेत तरी गोष्ट तुटत नाही: पण 3.8 (transformer, पूर्ण जोडून) मात्र वगळू नका: तो या section चा कळस आहे.

हे तुमच्या business decision मध्ये कुठे येईल: “आमचं product अर्थाने शोधतं” (semantic search), “आमच्याकडे embeddings आहेत”, “multimodal आहे”: अशा वाक्यांनी आज गुंतवणूक मागितली जाते आणि सेवा विकल्या जातात. या section नंतर ही वाक्यं तुमच्यासाठी धुकं नसतील: विकणाऱ्याला दोन नेमके प्रश्न विचारता येतील: आणि स्वतःच्या product मध्ये “अर्थ-शोध” कुठे लावायचा हे ओळखता येईल.

Chapter 3.1 [SPINE]: अर्थ numbers मध्ये बदलणं

Part 1 मधून एक अडचण उचलू जी तिथे बोलली गेली नव्हती. Token ला number मिळतो (1.6): समजा “चहा” = 48291 आणि “coffee” = 7102. हे numbers पत्ते आहेत: वहीतल्या नोंदींचे क्रमांक: आणि पत्त्यांमध्ये अर्थ शून्य आहे: 48291 आणि 7102 जवळ की दूर, या प्रश्नालाच अर्थ नाही. पण यंत्राला तर अर्थाने वागायचंय: “चहा घेणार का?” आणि “coffee घेणार का?” ही **जवळजवळ एकच** वाक्यं आहेत हे त्याला **कुठून** कळणार?

माणसाची जुनी युक्ती आठवा: **नकाशा**. गावांची नावं वेगळी असोत; नकाशावर **जागा** सांगते की कोण कोणाच्या शेजारी. मग अर्थाचाही नकाशा बनवू: प्रत्येक शब्दाला एक **जागा** द्यायची, अशी की **अर्थाने जवळचे शब्द जागेनेही जवळ**.

जागा म्हणजे काय, ते सोप्या पायरीने: एक शब्द अनेक **सरकपट्ट्यांवर** मोजा: किती सजीव (0-1)? किती गोड? किती महाग? किती औपचारिक?... “चहा”: सजीव 0, गोड 0.6, महाग 0.2... अशा शंभर पट्ट्या घेतल्या तर प्रत्येक शब्द = शंभर आकड्यांची यादी = शंभर-मिती जागेतला एक **बिंदू**. आणि दोन बिंदूंमधलं **अंतर** = अर्थामधलं अंतर: “चहा” आणि “coffee” चे बहुतेक आकडे जुळतील (जवळ); “चहा” आणि “truck” चे नाही (दूर). या आकड्यांच्या यादीचं नाव **vector**, आणि शब्दाला त्याची जागा देण्याच्या या रूपांतराचं नाव **embedding**: Book 2 चा सगळ्यात कामाचा नवा शब्द.

पण खरी जादू पुढे आहे: **या पट्ट्या कोणी design करत नाही**. “गोडपणा-पट्टी ठेवू” असं कोणी ठरवलं नाही. मग जागा कुठून येतात? Part 1 च्या चक्रातूनच (2.3): पुढचा-token ओळखायच्या सरावात यंत्राला जे शब्द **सारख्याच संगतीत** दिसतात (“गरम ___ प्या”: इथे चहा-coffee-दूध सगळे बसतात), त्यांना जवळच्या जागा दिल्या की अंदाज सुधारतो: म्हणून उतार (2.6) त्यांना आपोआप जवळ **सरकवतो**. जुनी म्हण यंत्राने पुन्हा शोधली: **शब्दाची ओळख त्याच्या संगतीने**. आणि पट्ट्यांचे अर्थ (“ही गोडपणाची, ती औपचारिकपणाची”) कोणालाच माहीत नसतात: त्या **घडलेल्या** असतात, design केलेल्या नाही (2.4 चं मडकं, पुन्हा).

या एका कल्पनेने एक जुनं दार उघडलं: **अर्थाने शोधणं**. Book 1 चा index (6.2) **नेमके शब्द** शोधायचा: “tiffin” शोधलं तर “tiffin” च सापडणार; “डबा-सेवा” लिहिलेली नोंद हरवणार. Embedding- शोध वेगळा: प्रश्नाचा vector काढा, **जवळचे** vectors शोधा: शब्द वेगळे असोत, अर्थ जुळला की नोंद सापडते. Google हेच करतं (“स्वस्त phone” शोधा: “budget smartphone” ची पानं येतात), YouTube-Spotify च्या शिफारसी हेच (“जवळची” गाणी), आणि तुमच्या gallery चं “beach” शोधणंही (Book 1, 6.2 च्या शेवटी भेटलेलं कोडं: आता सुटलं: photo चाही vector निघतो: chapter 3.10).

इथे लोक काय चुकीचं समजतात

”यंत्राला शब्दांचे अर्थ ‘कळतात’ म्हणजे dictionary आहे आत.” आता नेमकं चित्र: dictionary नाही, **नकाशा** आहे: आणि नकाशावर व्याख्या नसतात, फक्त **जागा आणि अंतरं** असतात. यातून एक खरी मर्यादाही निघते: संगत सारखी = जागा जवळ: म्हणून “डॉक्टर” आणि “रोगी” ही जवळ (एकाच वाक्यांत वावरतात!), फक्त “डॉक्टर”-“वैद्य” नाही. जवळीक म्हणजे “समानार्थी” नव्हे; “एकाच जगातले.” नकाशा वाचताना ही तळटीप विसरू नका.

MAP वर

Embedding हा स्वतः विकला जाणारा माल आहे: API-पानांवर “embeddings” ची वेगळी, स्वस्त किंमत असते (मजकूर द्या, vectors घ्या: प्रति million tokens काही cents), आणि हे vectors साठवून “जवळचे शोधा” करणाऱ्या खास databases चा (vector database: Book 1, 6.3 च्या आकारांचा नवा भाऊ) एक अख्खा उद्योग उभा राहिला आहे. Founder- नजर: तुमच्याकडे जुन्या नोंदी-तक्रारी-कागदपत्रांचा ढीग असेल, तर “अर्थाने शोध” हे त्याचं पहिलं, स्वस्त, लगेच-दिसणारं AI-रूप आहे: model न शिकवता, फक्त नकाशावर मांडून.

स्वतः बघा (5 मिनिटं)

अर्थ-शोध विरुद्ध शब्द-शोध, प्रत्यक्ष: (1) WhatsApp मध्ये जुना message शोधा: नेमका शब्द आठवावाच लागतो (शब्द-index). (2) आता Google Photos मध्ये “अन्न” शोधा: जेवणाचे photos येतात, कुठेही “अन्न” न लिहिता (अर्थ-नकाशा). (3) आणि Google वर मराठीत काहीतरी शोधा: इंग्रजी पानंही योग्य ती येतात: भाषा ओलांडूनही अर्थ-जागा जुळते: कारण नकाशात “पाणी” आणि “water” शेजारी राहतात.

विचार करा

- (derivation) नकाशाची कल्पना ताणून बघा: “राजा” आणि “राणी” जवळ असतील हे उघड. पण नकाशात **दिशांनाही** अर्थ असू शकतो का? “राजा -> राणी” ही उडी आणि “काका -> काकी” ही उडी: यांच्यात काय नातं असावं? (उत्तर पुढच्या chapter चं दार आहे.)

उत्तर: दोन्ही उड्या **एकाच दिशेने** असाव्यात: दोन्हीत बदलतोय तो एकच गुण (पुरुष -> स्त्री), बाकी सगळं कायम. जर नकाशा अर्थाने घडला असेल, तर “समान अर्थ-बदल = समान दिशा” हे आपोआप यायला हवं: आणि खरंच येतं: हीच या क्षेत्रातली सगळ्यात प्रसिद्ध गंमत आहे: राजा - पुरुष + स्त्री ~ राणी, अंकगणिताने! नकाशात नुसत्या जागा नाहीत; **नाती दिशांमध्ये** कोरलेली आहेत: कोणीही न शिकवता. किती खोल, आणि कुठे तुटतं: पुढचा chapter.

Chapter 3.2 [SPINE]: King आणि queen जवळ का राहतात

मागच्या chapter चा शेवट एका गंमतीवर झाला; आज ती गंमत पूर्ण उघडू, कारण तिच्यात या क्षेत्राचं सगळ्यात मोठं आश्चर्य **आणि** सगळ्यात मोठा धोका, दोन्ही एकत्र राहतात.

गंमत पुन्हा, नीट: शब्द-नकाशात (embeddings) फक्त **जागा** अर्थपूर्ण नाहीत; **दिशाही** आहेत. “राजा” च्या बिंदूपासून “राणी” कडे जी उडी, तीच उडी “काका” पासून मारा: **”काकी” जवळ पोहोचता**. “दिल्ली” - “भारत” + “France” ~ “Paris”. “चाललो” - “चालणे” + “बोलणे” ~ “बोललो”: व्याकरणाची रूपंही एकाच दिशेने! म्हणजे नकाशात अदृश्य **दिशा-रेषा** कोरल्या गेल्या आहेत: एक लिंगाची, एक राजधानीची, एक भूतकाळाची... आणि आठवा (3.1): हे कोणीही design केलं नाही: फक्त “पुढचा token ओळख” च्या सरावातून **उगवलं**.

हे इतकं आश्चर्यकारक का, ते Book 1 च्या भाषेत मोजा: कोणीही व्याकरणाचे नियम लिहिले नाहीत (rules नाहीत: 2.1!), कोणीही “भारताची राजधानी” ची नोंद केली नाही (database नाही: Book 1, 6.1!): आणि तरी नियम-नोंदींसारखं **वागणारं** काहीतरी वजनांमध्ये घडलं. कारणही चक्रातच आहे: “काका आले आणि ___ म्हणाल्या” इथे पुढचा token नीट ओळखायचा तर “काकी” ची उडी **लागतेच**: जी रचना अंदाज सुधारते, ती उतार आपोआप कोरतो (2.6). **भाषेत जग मुरलेलं आहे; भाषेच्या अंदाजात जगाची रचना उगवते**. हीच LLM-युगाची मध्यवर्ती घटना आहे.

आता नाण्याची दुसरी बाजू, आणि ती हसण्यावारी नेऊ नका: **दिशा संगतीतून येतात, सत्यातून नाही** (3.1 ची तळटीप). जगाच्या लिखाणात जे कल आहेत, ते दिशांमध्ये **कोरले** जातात: याच नकाशांत संशोधकांना सापडलं: “doctor” ची जागा “he” कडे झुकलेली, “nurse” ची “she” कडे; काही समाजगट “गुन्हा”-शब्दांच्या शेजारी. नकाशाने जगाचं **वर्णन** केलं: पण जुन्या जगाचे पूर्वग्रह भूगोल बनून गोठले. आणि गोठलेला भूगोल **वापरात** आला की वर्णनाचं रूपांतर **निर्णयात** होतं: हाच धोका.

इथे लोक काय चुकीचं समजतात

“यंत्राला व्याकरण/भूगोल ‘समजला’ आहे.” नेमकं वाक्य: यंत्राकडे **नियम** नाहीत; नियमांसारख्या **सावल्या** आहेत: दिशांच्या रूपात. सावल्या बहुधा बरोबर पडतात, पण त्या सावल्याच आहेत: ओढून बघितलं की तुटतात (दुर्मिळ शब्द, नवी नाती, विचित्र जोड्या: अंकगणित नेहमी “जवळ” पोहोचवतं, “नेमकं” नाही). म्हणून भाषेच्या पोताजवळ यंत्रावर विश्वास ठेवा; **नियम म्हणून** त्याच्या सावल्यांवर इमारत बांधू नका. (आणि उलटी चूकही नको: “सावल्याच आहेत, म्हणजे कचरा”: सावल्यांनी भाषांतर-शोध-शिफारस ही अख्खी साम्राज्यं चालवली आहेत.)

MAP वर

पूर्वग्रह-दिशा ही आता कायद्याची आणि पैशाची बाब आहे: नोकर-भरती गाळणीत AI वापरून अमेरिकेत खटले-दंड झाले आहेत; EU चा AI-कायदा “उच्च-धोका” वापरांना (भरती, कर्ज, वैद्यकीय) कडक तपासणी लावतो; भारतातही ही चौकट येते आहे. म्हणजे “आमच्या यंत्रात कुठले कल आहेत हे आम्ही मोजलं” ही ओळ आता विक्री-कागदांची गरज आहे: आणि ती **मोजणारा** एक छोटा उद्योगही उभा राहतोय (bias-audit: 2.8 च्या चाळणी-तपासणीची बहीण). तुमच्या product मध्ये AI कोणाबद्दल **निर्णय** घेत असेल (ग्राहक निवड, किंमत, प्राधान्य), तर हा chapter तुमचा कायदे-सल्लागार आहे: निर्णयाआधी माणूस ठेवा, आणि कल मोजून ठेवा.

स्वतः बघा (5 मिनिटं)

सावल्या आणि त्यांची दुरुस्ती, दोन्ही एकदम: Claude/ChatGPT ला सांगा: “एका doctor बदल दोन ओळींची गोष्ट लिही, आणि एका nurse बदलही.” लिंग कोणतं आलं ते बघा. मग विचारा: “उलट लिंगांनी पुन्हा लिही.” आजची यंत्रं बहुधा सहज पुन्हा लिहितील: आणि काही तर पहिल्याच फटक्यात जाणीवपूर्वक तटस्थ राहतील: ती 2.9 ची शिकवणी जुन्या दिशांना **झाकते** आहे. दोन थर एकाच वेळेस दिसले: खाली गोठलेला भूगोल, वर घातलेला लगाम: हेच आजचं AI आहे.

विचार करा

1. (derivation) “भाषेच्या अंदाजात जगाची रचना उगवते”: हे सूत्र उलटं फिरवा: जगातली एखादी रचना अशी सांगा जी **भाषेत फारशी लिहिलीच जात नाही**: आणि म्हणून यंत्राच्या नकाशात धूसर असणार.

उत्तर: उदाहरणं भरपूर: हाताचं ज्ञान (कणीक “योग्य” मळली हे बोटांना कळतं: शब्दांत क्वचित), गावचा अलिखित व्यवहार (कोणाला उधारी द्यायची हे “चेहऱ्यावरून”), वास-चव-स्पर्श, आणि जे लिहिणं टाळलं जातं (अपयश, लाज, घरगुती हिशोब: 2.7 च्या रिकाम्या जागा). नियम असा निघतो: **यंत्राचा नकाशा = लिहिल्या-गेलेल्या जगाचा नकाशा**; न-लिहिलेलं जग त्याच्यासाठी समुद्रातलं न-मोजलेलं बेट. धंद्याची जोड: तुमच्या क्षेत्रातलं असं “न-लिहिलेलं ज्ञान” जर तुम्ही **लिहून/नोंदवून** काढलंत (2.3 चा data-चक्र!), तर तुमच्याकडे असा नकाशा-तुकडा येतो जो जगातल्या कुठल्याच मोठ्या यंत्राकडे नाही: छोट्या माणसाची मोठी जागा तीच आहे.

Chapter 3.3 [SPINE]: एका शब्दाचे अनेक अर्थ

नकाशाची कल्पना (3.1) आता एका खडकावर आपटवू कारण त्या धडकेतूनच पुढची मोठी कल्पना जन्मते.

खडक हा: **”पान.”** झाडाचं पान. पुस्तकाचं पान. विड्याचं पान. जेवणाचं पान (वाढलेलं ताट). एकच token, चार जगं. आता 3.1 चा नकाशा आठवा: प्रत्येक शब्दाला **एक** जागा. “पान” चा बिंदू कुठे ठेवणार? झाडांच्या वस्तीत ठेवला तर पुस्तक हरवलं; चारही वस्त्यांच्या **मध्ये** ठेवला तर... तो कुठल्याच वस्तीत नाही: सगळ्या अर्थांची मिळून एक पाणचट सरासरी. (इंग्रजीचं गाजलेलं उदाहरण: “bank”: नदीकाठ की पैशाची पेढी: same खडक.)

माणूस हे कोडं कसं सोडवतो? क्षणात, आणि नकळत: **शेजार बघून**. “विड्याचं पान तोंडात टाकलं” इथे “विड्याचं” आणि “तोंडात” हे शेजारी “पान” चा अर्थ **ठरवतात**. म्हणजे खरा नियम असा: **शब्दाचा अर्थ शब्दात नसतो; वाक्यात असतो**. तोच माणूस घरात “बाबा”, शाळेत “सर”, दुकानात “शेठ”: ओळख तीच, **भूमिका** संदर्भाने.

मग यंत्रासाठी उपाय आपोआप दिसतो: शब्दाची जागा **गोठलेली** ठेवायचीच नाही. सुरुवात नकाशाच्या सामान्य जागेपासून करा (“पान” ची मधली, पाणचट जागा): आणि मग **वाक्य वाचता-वाचता ती जागा सरकू द्या**: “विड्याचं” शेजारी दिसताच “पान” चा बिंदू विड्याच्या वस्तीकडे **सरकावा**; “पुस्तकाचं” दिसताच पुस्तकांच्या वस्तीकडे. प्रत्येक शब्दाची अंतिम जागा = मूळ जागा + शेजाऱ्यांनी दिलेले धक्के. याला म्हणतात **contextual embedding**: संदर्भाने घडणारी जागा: आणि आजची सगळी मोठी यंत्रं फक्त अशाच, सरकत्या जागा वापरतात.

पण थांबा: “शेजाऱ्यांनी धक्के द्यावेत” हे बोलायला सोपं आहे. लगेच तीन काटेरी प्रश्न उभे राहतात:

कोणी धक्का द्यायचा? सगळ्याच शेजाऱ्यांनी नाही: “विड्याचं पान तोंडात टाकलं आणि गाडीकडे निघाला” इथे “गाडी” ने “पान” ला धक्का देणं **चूक** ठरेल. म्हणजे प्रत्येक शब्दाने **निवडायचं** आहे: माझ्यासाठी कोण महत्त्वाचा.

किती जोराचा? “विड्याचं” चा धक्का जोरदार; “आणि” चा जवळजवळ शून्य. म्हणजे धक्के **वजनदार** हवेत: कोणाचा किती, हे आकड्यात.

लांबच्याने कसा द्यायचा? “जे पान त्याने सकाळी बाजारातून आणलं होतं, ते संध्याकाळी त्याने तोंडात टाकलं”: इथे “तोंडात” आणि “पान” मध्ये दहा शब्दांचं अंतर आहे: तरी धक्का पोहोचलाच पाहिजे.

ही तीन प्रश्नांची यादी नीट बघा: ”प्रत्येक शब्दाने, पूर्ण वाक्यातून, आपल्याला हवे ते शेजारी निवडून, वजनदार धक्के घेणं”: या यंत्रणेचं नाव आहे **attention**. नाव तुम्ही ऐकलं आहे; आता गरज का आहे हे तुमच्या हातात आहे: आणि पुढचे दोन chapters (3.4, 3.5) ही यंत्रणा आतून उघडतात.

इथे लोक काय चुकीचं समजतात

”यंत्राला ‘पान’ चे चार अर्थ list करून दिले असतील.” आता तुम्हाला माहीत आहे: कुठलीही list नाही (rules नाहीत: 2.1!). आहे एक सामान्य जागा + सरकण्याची यंत्रणा: अर्थ **प्रत्येक वाक्यात नव्याने घडतो**. याचा एक सुंदर परिणाम: यंत्र **नवे** अर्थही हाताळतं: “battery चं पान” असं कोणी लिहिलं (नवीन वापर!), तर list-पद्धत हरली असती; सरक-पद्धत शेजाऱ्यांवरून जागा जुळवते. आणि एक मर्यादाही: संदर्भच फसवा असेल तर जागा चुकीची सरकते: विनोद, श्लेष, उपरोध इथे यंत्र घसरतं ते म्हणूनच.

MAP वर

”संदर्भाने अर्थ” ही कल्पना विक्रीयोग्य फरक आहे: जुना keyword- शोध “पान” शोधल्यावर चारही जगांचा कचरा आणतो; संदर्भ-जाणणारा शोध “विड्याची पानं घाऊक” ही मागणी नेमकी पोहोचवतो. ग्राहक-सेवेत हाच फरक पैसा आहे: “खातं बंद पडलं” (bank-app मधली तक्रार) आणि “खातं बंद करायचंय” (सोडून जाणारा ग्राहक!); शब्द जवळजवळ तेच, अर्थ उलटे, आणि दुसऱ्याला **आधी** ओळखणं म्हणजे गळणारा ग्राहक वाचवणं. संदर्भ वाचणारी यंत्रणा हेच ओळखते: keyword-यंत्रणा दोन्हींना एकच रांग लावते.

स्वतः बघा (5 मिनिटं)

यंत्राची सरकती जागा प्रत्यक्ष बघा: Claude/ChatGPT ला तीन प्रश्न, वेगवेगळ्या chats मध्ये: (1) “पान म्हणजे काय?” (मधली, पाणचट जागा: उत्तर बहुधा “अनेक अर्थ आहेत” ने सुरू होईल: नकाशाची सरासरी!) (2) “जेवायला बसलो, पान वाढलं: इथे पान म्हणजे?” (3) “पान 42 वर काय आहे हे कसं बघू?”: प्रत्येक वेळेस यंत्र नेमक्या वस्तीत पोहोचेल. तुम्ही आता contextual embedding चं वागणं बाहेरून मोजलंत.

विचार करा

1. (derivation) “शब्दाचा अर्थ वाक्यात असतो” हे सूत्र एक पायरी ताणा: **वाक्याचा** अर्थ कशात असतो? आणि याचा तुमच्या AI-वापरावर थेट परिणाम काय?

उत्तर: वाक्याचा अर्थ **परिच्छेदात/संवादात** असतो: “हो, चालेल” या वाक्याचा अर्थ मागच्या प्रश्नाशिवाय शून्य आहे. हीच शिडी वर जाते: शब्द <- वाक्य <- संवाद <- परिस्थिती. परिणाम: यंत्राला प्रश्न देताना **संदर्भ देणं** हे ऐच्छिक नाही; तेच अर्थाचं इंधन आहे: “हे चुकलय, नीट कर” पेक्षा “टिफिन-app च्या refund-नियमाचा हा मसुदा, ग्राहक रागावलेला आहे, नम्र पण ठाम उत्तर हवं” हे दहापट चांगलं उत्तर देतं: कारण तुम्ही यंत्राच्या जागा **योग्य वस्तीत सरकवून** दिल्या. Part 3 मध्ये या कलेचं पूर्ण शास्त्र (prompting) आहे: पण मूळ नियम हाच: **अर्थ हवा तर संदर्भ द्या.**

Chapter 3.4 [SPINE]: कुठला शब्द कोणावर अवलंबून

मागच्या chapter ने attention ची गरज सिद्ध केली: प्रत्येक शब्दाने वाक्यातून आपले महत्वाचे शेजारी निवडायचे. आज ही निवड किती अवघड आहे ते मोजू: आणि जुनी पद्धत या ओझ्याखाली का मेली तेही: कारण त्या मृत्यूतूनच transformer जन्मला.

आधी अवघडपणा डोळ्यांनी बघा. पुढचा शब्द ओळखायचा आहे:

”जो मुलगा काल संध्याकाळी आपल्या दुकानात ती निळी छत्री विसरून गेला होता, तो आज सकाळी परत _____”

रिकाम्या जागेला काय हवं? “आला.” आणि “तो” म्हणजे कोण हे ठरवायला कुठला शब्द लागला?

”मुलगा”: वाक्याच्या सुरुवातीचा, सोळा शब्द मागचा. मधले “छत्री”, “दुकान”, “संध्याकाळ” सगळे जवळ आहेत पण या क्षणी **बिनमहत्वाचे**. म्हणजे नियम क्रूर आहे: **महत्त्व अंतरावर अवलंबून नाही**. जवळचा शब्द निरुपयोगी असू शकतो; लांबचा जीवनावश्यक. आणि कोण महत्वाचा हे **प्रत्येक रिकाम्या जागेला नव्याने** ठरतं: पुढच्याच क्षणी “छत्री” राणी बनेल (“...परत आला आणि आपली _____ मागितली”).

आता जुनी पद्धत बघा, जिने 2017 पर्यंत राज्य केलं: वाक्य **क्रमाने, एक-एक शब्द** वाचायचं, आणि वाचता-वाचता एक “आठवणीची पिशवी” (एक ठराविक आकाराचा vector) सोबत न्यायची: प्रत्येक नवा शब्द पिशवीत मिसळायचा. (कुळाचं नाव: RNN: recurrent, म्हणजे फिरून-फिरून.) कल्पना नैसर्गिक वाटते: माणूसही डावीकडून उजवीकडे वाचतो ना? पण दोन जीवघेणी दुखणी:

दुखणं 1: पिशवी गळते. सोळा शब्द मिसळल्यावर पहिल्या शब्दाचं काय उरतं? निरोप्या-साखळीचा खेळ आठवा (एकाने दुसऱ्याला, दुसऱ्याने तिसऱ्याला...): दहाव्या टप्प्यावर निरोप ओळखू येत नाही. “मुलगा” पिशवीत होता: पण चौदा मिसळण्यांनी **धूसर** झाला. लांबचं नातं = गळलेली आठवण: आणि भाषा लांबच्या नात्यांनीच बांधलेली आहे.

दुखणं 2: रांग मोडता येत नाही. शब्द 16 मिसळण्याआधी शब्द 15 संपलाच पाहिजे: काम मुळातच **क्रमिक** आहे. आणि Book 1 ची आठवण काढा (GPU: हजारो हात, एकदम: 2.6): क्रमिक कामाला हजार हात दिले तरी 999 रिकामे बसतात! Data चा महापूर आणि GPU ची फौज दारात उभी: पण रचनाच रांगेची: शाळा मोठी करता येईना.

दोन दुखणी एकत्र वाचा, आणि उपाय जवळजवळ स्वतः सुचतो: **पिशवी फेकून द्या. रांगही फेकून द्या.** प्रत्येक शब्दाला **सगळ्या** शब्दांकडे **थेट** बघू द्या: “मुलगा” सोळा शब्द मागे असो: “तो” ची नजर थेट त्याच्यावर: मधल्या चौदा मिसळण्यांची साखळी नाहीच, म्हणून गळणंही नाही. आणि प्रत्येक शब्दाची ही

“सगळ्यांकडे बघणी” दुसऱ्या शब्दाच्या बघणीवर अवलंबून नाही: म्हणजे **सगळ्या बघण्या एकाच वेळेस**: GPU ची फौज पूर्ण कामाला (Book 1, 7.1 चं आडवं- वाढणं, वाक्याच्या आत!).

“थेट, सगळ्यांकडे, एकदम बघणं: आणि बघताना प्रत्येकाला वजन देणं”: याचं नाव attention, हे मागच्या chapter अखेर कळलं होतंच. आता त्याची जन्मकथाही तुमच्याकडे आहे: **attention ही फक्त हुशारीची कल्पना नाही; ती गळक्या पिशवीवरची आणि रिकाम्या GPU वरची, दुहेरी दुरुस्ती आहे.** 2017 च्या ज्या शोधनिबंधाने हे मांडलं, त्याचं नाव आता गंमतीदार वाटेल: “Attention Is All You Need”: पिशवी- रांग काहीच नको; बघणं पुरे. पुढचा chapter (DEPTH) ते बघणं आतून उघडतो: वजन नेमकी कशी ठरतात.

इथे लोक काय चुकीचं समजतात

“यंत्र माणसासारखं डावीकडून उजवीकडे वाचत असेल.” आता तुम्हाला माहीत आहे: आजचं यंत्र वाक्य **एकदम, चौफेर** बघतं: वाचत नाही, **न्याहाळतं**: नकाशा बघावा तसं. (माणूसही खरं तर उड्या मारत, मागे-पुढे करत वाचतो: डोळ्यांच्या हालचाली मोजणाऱ्यांना हे जुनं माहीत आहे.) आणि दुसरी गल्लत: “जुनी पद्धत ‘चुकीची’ होती.” नव्हती: छोट्या वाक्यांवर RNN उत्तम चालायची: ती **scale** वर मेली: गळकी पिशवी + रिकामी GPU-फौज. Book 1, 7.6 चा धडा यंत्र-रचनेतही खरा: bottleneck रचनेत असला की मोठेपण विकत घेता येत नाही.

MAP वर

या chapter मधला व्यापारी धडा रचनेच्या निवडीबद्दल आहे: RNN -> transformer ही उडी “10% सुधारणा” नव्हती; “**GPU-फौजेला पूर्ण जुंपता येणं**” ही होती: आणि त्याच क्षणी data-केंद्रांचा, Nvidia चा, frontier-खर्चाचा (\$100M-\$1B: 2.6) आजचा खेळ शक्य झाला. नियम लक्षात ठेवा: **जी रचना उपलब्ध हत्याराशी (इथे: समांतर GPU) जुळते, ती “थोडी वाईट” असूनही जिंकते; जी जुळत नाही, ती “हुशार” असूनही मरते.** Founder-रूप: product/process design करताना विचारा: माझी रचना माझ्या हत्यारांच्या (टीम, यंत्र, भांडवल) धाटणीशी जुळते का: की मी क्रमिक साखळी बांधून हजार हातांना रिकामं बसवतोय?

स्वतः बघा (5 मिनिटं)

यंत्राची “थेट लांब नजर” तपासा: Claude/ChatGPT ला वीस-पंचवीस ओळींची एक गोष्ट द्या (कुठलीही), जिच्या **पहिल्या** ओळीत एक बारीक तपशील पेरा (“...त्याच्या खिशात हिरवं पेन होतं...”). मग शेवटी विचारा: “पेनाचा रंग काय होता?” उत्तर क्षणात, बिनचूक येईल: सोळाच काय, हजार शब्दांवरूनही: कारण

नजर थेट आहे, साखळी नाही. (आणि याच प्रयोगाची मर्यादाही पुढे भेटेल: रांग **खूपच** लांब झाली की मधलं हरवायला लागतं: chapter 4.5 ची चाहूल.)

विचार करा

1. (derivation) “प्रत्येक शब्द सगळ्यांकडे बघतो” या design ची एक **किंमत** असणारच (या पुस्तकाची सवय!). शब्द 100 असतील तर बघण्या किती? 1,000 असतील तर? कुठला जुना राक्षस परत आला: आणि याचा एक रोजचा परिणाम सांगा.

उत्तर: प्रत्येकाची प्रत्येकाशी = 100 शब्दांना ~10,000 बघण्या; 1,000 शब्दांना ~1 million: **रांग दहापट, काम शंभरपट:** Book 1, 5.1 चा n -squared राक्षस, यावेळेस वाक्याच्या आत! रोजचा परिणाम: लांब संवाद/मोठी कागदपत्रं देताच यंत्र हळू आणि **महाग** होतं (context ची किंमत pricing-पानांवर वेगळी दिसते ती म्हणूनच), आणि खिडकीला मर्यादा असते (1-2 million tokens: 2.10) तीही म्हणूनच: अमर्याद खिडकी = अमर्याद वर्ग-वाढ. या राक्षसाला काबूत ठेवण्याच्या युक्त्या (स्वस्त-attention चे प्रकार, cache: 4.2) हा आजच्या संशोधनाचा जिवंत रणांगण आहे: आणि आता त्या बातम्याही तुम्हाला वाचता येतील.

Chapter 3.5 [DEPTH]: Attention आतून

(DEPTH chapter. AI-जगातला सगळ्यात गाजलेला शब्द आतून बघायचा आहे. जड नाही: एक बाजार-दृश्य पुरेल: पण वगळायचा असेल तर एवढं घेऊन पुढे जा: “प्रत्येक शब्द प्रश्न विचारतो, बाकीचे उत्तरं देतात, आणि जुळणीच्या प्रमाणात मिसळण होते.”)

3.3 ने मागणी लिहिली होती: कोण? किती जोरात? लांबूनही? आता यंत्रणा. दृश्य: आठवडी बाजार. प्रत्येक शब्द म्हणजे एक माणूस: आणि गंमत अशी की प्रत्येकजण एकाच वेळेस **गिन्हाईकही** आहे आणि **दुकानदारही**.

प्रत्येक शब्दाकडून तीन कागद बनवले जातात (कसे: त्याच्या vector ला तीन वेगळ्या, शिकलेल्या वजनांनी गुणून: 2.4 ची वजनं इथे कामाला):

कागद 1: मागणी (query). “मला काय हवंय?” “तो” चा कागद: “मी सर्वनाम आहे; मला माझा मालक हवाय: पुरुषी, एकवचनी, माणूस.”

कागद 2: पाटी (key). “माझ्याकडे काय आहे?” “मुलगा” ची पाटी: “मी आहे: पुरुषी, एकवचनी, माणूस, वाक्याचा कर्ता.”

कागद 3: माल (value). पाटी जुळल्यावर प्रत्यक्ष **काय द्यायचं** ते: “मुलगा” चा माल: त्याच्या अर्थाचा गूठा (काल आलेला, छत्री विसरलेला...).

आता बाजार भरतो: **प्रत्येक गिन्हाईकाची मागणी x प्रत्येक दुकानाची पाटी**: जुळणीचा एक आकडा (score). “तो” ची मागणी “मुलगा” च्या पाटीशी घट्ट जुळते: मोठा आकडा; “छत्री” शी जुळत नाही: छोटा. मग सगळे आकडे **हिश्यांमध्ये** बदलले जातात (बेरीज 1: probability ची जुनी पट्टी: 1.3!): “मुलगा” 0.85, “दुकान” 0.06, “छत्री” 0.03... आणि शेवटी गिन्हाईक प्रत्येक दुकानातून **हिश्याच्या प्रमाणात माल** घेऊन मिसळतं: 85% “मुलगा” चा अर्थ-गूठा, चिमूटभर बाकीचे. निकाल: “तो” चा **नवा** vector: आता त्याच्यात “मुलगा” मुरलेला आहे: 3.3 चं “सरकणं” हे **हेच**.

तीन बारकावे, तीन जुन्या प्रश्नांची उत्तरं:

”**कोण ठरवतं जुळणी?**” कोणीच नाही: मागणी-पाटी बनवणारी वजनं **शिकलेली** आहेत (2.3 चं चक्र): कोट्यवधी वाक्यांत “कुठली बघणी अंदाज सुधारते” या एकाच कसोटीने घासलेली. व्याकरणाचा board कुठेच नाही: पण “सर्वनामाने कर्त्याकडे बघावं” हे **उपयोगी** निघालं, म्हणून उगवलं (3.2 च्या सावल्यांचं भावंडं).

”लांबचा कसा?” बाजारात सगळी दुकानं एकाच पटांगणात: “मुलगा” सोळा जागा मागे असो: त्याची पाटी तितकीच जवळ. अंतर या यंत्रणेला **दिसतच नाही** (आणि त्यामुळे एक नवं कोडं जन्मतं: क्रम कसा कळणार? chapter 3.7 चं काम).

”एकदम कसं?” सगळ्या मागण्या $\times$ सगळ्या पाट्या = एक राक्षसी गुणाकार-तक्ता: आणि हे नेमकं GPU चं जन्म-काम (2.6 ची शाळा): म्हणून बाजार **एका क्षणात** भरतो: n-squared किंमत मोजून (3.4 चा राक्षस), पण समांतर.

इथे लोक काय चुकीचं समजतात

”Attention म्हणजे यंत्र ‘लक्ष देतं’: माणसासारखं, जाणीवपूर्वक.” आता आतलं दृश्य तुमच्याकडे आहे: लक्ष नाही, **गुणाकार** आहे: मागणी-पाटीची जुळणी, हिस्से, मिसळण. नाव मानवी आहे; यंत्रणा बाजारी आहे. पण उलटी टोकाची चूकही नको: “नुसता गुणाकार आहे, म्हणजे पोकळ आहे”: या “नुसत्या” गुणाकाराने गळकी पिशवी आणि रांग: दोन्ही मारल्या (3.4), आणि भाषेची लांब नाती पहिल्यांदा यंत्राच्या हातात दिली. साधी यंत्रणा $\times$ राक्षसी प्रमाण = खोल वागणं: Book 1, 1.1 पासूनची या दोन्ही पुस्तकांची एकच गाणी.

MAP वर

हा बाजार चालवण्याची किंमत हाच आजच्या AI-अर्थकारणाचा engine- खर्च आहे: प्रत्येक नव्या token ला मागच्या **सगळ्या** पाट्यांशी जुळणी (म्हणून लांब context महाग: 3.4), आणि ही जुळणी GPU वर (म्हणून Nvidia: \$5.5 trillion). पण एक चलाखी लक्षात घ्या: जुन्या tokens च्या पाट्या-माल **बदलत नाहीत**: मग त्या पुन्हा कशाला मोजायच्या? साठवून ठेवा: Book 1, 6.6 चा cache, इथे “KV-cache” नावाने (पाट्या=K, माल=V!): आणि API-pricing मधली “cache hit: 10% किंमत” ही ओळ (Part 1 ची) आता तुम्हाला **आतून** कळते: पाट्या पुन्हा न मोजण्याची सूट आहे ती. Chapter 4.2 मध्ये हा हिशोब पूर्ण होईल.

स्वतः बघा (5 मिनिटं)

बाजार-यंत्रणा स्वतःवर चालवा. हे वाक्य वाचा: “शिक्षकांनी मुलांना ओरडून सांगितलं की **त्यांचा** निकाल उद्या लागेल.” “त्यांचा” म्हणजे कोणाचा: शिक्षकांचा की मुलांचा? क्षणभर थांबलात ना: आणि मग “निकाल” या शेजाऱ्याने पारडं “मुलांकडे” झुकवलं? तुम्ही आता स्वतःची मागणी-पाटी जुळणी **जाणवून** घेतली: आणि जिथे माणूस थांबतो, तिथे यंत्रही डळमळतं (दोन्ही पारडी जवळ: 0.55-0.45!): अस्पष्ट संदर्भ = अस्पष्ट attention = डळमळीत उत्तर. लिहिताना संदिग्धता टाळा: हा फक्त भाषेचा नाही, यंत्र-वापराचाही नियम झाला.

विचार करा

1. (derivation) मागणी-पाटी-माल हे **तीन वेगळे** कागद का? “पाटी हाच माल” ठेवला असता (जुळणीही त्याच्यावर, देणही तेच) तर काय बिघडलं असतं? (बाजाराच्या उपमेतच विचार करा.)

उत्तर: दुकानाची **पाटी** आणि दुकानातला **माल** ही कामं मुळात वेगळी आहेत: पाटी म्हणते “मी कशाशी जुळतो” (शोधण्यासाठी ठळक, मोजकं); माल म्हणतो “मी प्रत्यक्ष काय देतो” (समृद्ध, सगळं). एकच ठेवला तर एक तडजोड: शोधायला सोपं ठेवावं तर देणं गरीब होतं; देणं श्रीमंत ठेवावं तर जुळणी गढूळ. वेगळे कागद = प्रत्येक कामाला मोकळं शिकता येतं (“कशावरून सापडावं” वेगळं, “काय द्यावं” वेगळं). हीच कल्पना तुम्ही Book 1 मध्ये भेटला आहात: index वेगळा, नोंद वेगळी (6.2: यादीत फक्त शोध-चावी + खूण; माल मूळ जागी)! चांगल्या रचना पुन्हा-पुन्हा तीच गोष्ट सांगतात: **शोधण्याचं रूप आणि देण्याचं रूप वेगळं ठेवा.**

Chapter 3.6 [DEPTH]: एकाच वेळेस अनेक गोष्टी बघणं

(DEPTH chapter. छोटा आहे: मागच्या बाजाराला एक मजला चढवायचा आहे, आणि “multi-head” हा pricing-पानांवरचा शब्द उघडायचा आहे.)

मागच्या chapter च्या बाजारात एक अडचण लपली आहे. “तो” ची मागणी आपण लिहिली: “माझा मालक हवाय.” पण त्याच क्षणी “तो” ला अजून काही हवं असतं: क्रियापदाशी जोड (“आला” शी वचन-लिंग जुळवायचंय), भावनेचा सूर (गोष्ट प्रेमळ आहे की रागीट?), विषयाची वस्ती (दुकान-जग की शाळा-जग?)... **एकाच शब्दाच्या एकाच क्षणी अनेक, वेगवेगळ्या भुका आहेत.** एका मागणी-कागदात हे सगळं कोंबलं तर? सगळ्या भुकांची मिळून एक गढूळ सरासरी: ना धड मालक सापडेल, ना धड सूर (3.3 च्या “पाणचट मध्यबिंदू”चीच पुनरावृत्ती).

उपाय बाजारातलाच: **एक गिऱ्हाईक अनेक याद्या घेऊन फिरतं.** भाजीची यादी वेगळी, किराणा वेगळा, कापड वेगळं: आणि चतुर घरात कामं वाटलेली: एकजण भाजी-रांगेत, दुसरा किराणा-रांगेत, **एकाच वेळेस.** यंत्रात नेमकं हेच: attention चा बाजार **एकदा नाही, अनेकदा** भरतो: समांतर: प्रत्येक फेरीचे स्वतःचे मागणी-पाटी-माल कागद (स्वतःची शिकलेली वजनं), म्हणजे प्रत्येक फेरी **वेगळ्या प्रकारचं नातं** धुंडाळायला मोकळी. एका फेरीचं नाव **head**; अनेकांचं: **multi-head attention**. आजच्या मोठ्या यंत्रांत एका थरात 32-128 heads असतात.

आणि heads काय-काय शिकतात? आधीच ठरवलेलं काहीच नाही (या पुस्तकाची सवय: design नाही, उगवतं): पण घडलेली यंत्रं उघडून पाहणाऱ्या संशोधकांना सापडलं: कुठलं head सातत्याने “मागच्या शब्दाकडे” बघतं, कुठलं “वाक्याच्या सुरुवातीकडे”, कुठलं सर्वनाम-मालक जोड्यांकडे, कुठलं स्वल्पविराम-रचनेकडे: आणि बरीच heads अशी की त्यांचं काम माणसाच्या कुठल्याच शब्दात बसत नाही. पुन्हा मडकं (2.4): भूमिका **घडतात**, वाटल्या जात नाहीत.

एक शेवटची जोडणी, जिच्यामुळे हा chapter फक्त गंमत नाही: **heads समांतर आहेत:** एकमेकांची वाट कोणी बघत नाही: म्हणजे GPU-फौजेला अजून एक मजला काम (3.4 चा धडा: रचना हत्याराशी जुळली की जिंकते). Multi-head हा “हुशारीचा” शोध आहेच: पण तो **समांतरतेचाही** शोध आहे: एकाच किमतीत अनेक नजरा.

इथे लोक काय चुकीचं समजतात

”जास्त heads = जास्त हुशार.” थेट नाही: heads म्हणजे **नजरांचे प्रकार**, आणि प्रकारांची गरज कामावर ठरते (Book 1, 3.4 ची गाडी-निवड!). संशोधनात हेही सापडलं: घडलेल्या यंत्रातली काही heads **काढून टाकली** तरी फरक जेमतेम पडतो: कामाची वाटणी विषम असते, काही नजरा जवळजवळ रिकाम्या. (छोटी-स्वस्त यंत्रं बनवणारे नेमकं हेच छाटतात.) मोजपट्टी “किती heads” नाही; “कुठली नाती पकडली जातात” ही आहे: आणि ती बाहेरून वागण्यातच दिसते.

MAP वर

”Multi-head, layers, parameters” ही यंत्र-वर्णनाची भाषा गुंतवणूक-पानांवर आणि तांत्रिक मुलाखतींमध्ये फेकली जाते: आता तुम्ही ती **तोळू** शकता: heads = समांतर नजरा; जास्ती असणं हे मोठेपणाचं लक्षण, हुशारीची हमी नाही. आणि एक व्यापारी समांतर घ्या: तुमच्या धंद्यातही “एकाच गोष्टीकडे अनेक स्वतंत्र नजरांनी बघणं” हीच quality-रचना आहे: तक्रारीकडे एक नजर पैशाची, एक भावनेची, एक कायद्याची: तिन्ही **स्वतंत्र**, मग मिसळण. (आणि हो: हे पुस्तक ज्या तपासण्यांतून जातं त्यातही हेच: मजकूर- जुळणी वेगळी, रचना-तपासणी वेगळी, डोळ्यांनी बघणं वेगळं.)

स्वतः बघा (5 मिनिटं)

स्वतःची multi-head चालवून बघा. हे वाक्य **एकदाच** वाचा: “काल रात्री उशिरा आलेल्या पाहुण्यांनी आईने केलेली खीर संपवली.” आता न बघता उत्तरं द्या: (1) कोण आलं? (2) कधी? (3) खीर कोणी केली? (4) वाक्याचा सूर तक्रारीचा की कौतुकाचा? चारही उत्तरं आली: म्हणजे एका वाचनात तुमच्या चार नजरांनी चार वेगळी नाती उचलली: कर्ता-नजर, वेळ-नजर, मालकी-नजर, सूर-नजर. यंत्राची heads हीच वाटणी करतात: फक्त 32-128 नजरांनी, एकाच क्षणात.

विचार करा

- (derivation) “सगळ्या heads ना सारखंच शिकू दिलं तर?” म्हणजे सुरुवातीची वजनं सगळ्यांची **एकसारखी** ठेवली तर: चक्र (2.3) चालल्यावर काय घडेल, आणि यातून एक सामान्य नियम काढा.

उत्तर: एकसारखी सुरुवात = एकसारखे उतार = **कायम एकसारखीच** वजनं: 64 heads म्हणजे प्रत्यक्षात एकच नजर 64 वेळा छापलेली: वाटणीच जन्मणार नाही. म्हणून सुरुवात **मुद्दाम random** असते (2.3 मधला “अडाणी” जन्म!): वेगळे जन्म-बिंदू -> वेगळे उतार -> वेगळ्या भूमिका उगवतात. सामान्य नियम: **विविधता हवी असेल तर सुरुवातीचं वेगळेपण राखा; सगळ्यांना एकाच साच्यातून काढलं तर फौज मिळते, नजरा नाहीत.** (माणसांच्या team बदलही हे वाचता येतं: पण तो निष्कर्ष तुमच्यावर सोडला.)

Chapter 3.7 [DEPTH]: क्रम कसा कळतो

(DEPTH chapter, आणि सगळ्यात छोटा: पण 3.5 ने मुद्दाम उघडं ठेवलेलं एक भोक बुजवायचं आहे: नाहीतर transformer ची इमारत (3.8) एका पायावर लंगडी राहील.)

भोक आठवा: attention च्या बाजारात **अंतर दिसत नाही** (3.5): सगळ्या पाट्या एकाच पटांगणात, जुळणी फक्त मजकुरावर. सोय होती ती (लांबचा शब्द जवळच्याइतकाच!): पण किंमत भयंकर आहे: बाजाराला **क्रमच** दिसत नाही. “रामाने रावणाला मारलं” आणि “रावणाने रामाला मारलं”: शब्द तेच, फक्त जागा बदलल्या: आणि बाजाराच्या नजरेत दोन्ही वाक्यं **एकसारखी!** भाषेत अर्थाचा अर्धा भार क्रमावर आहे: आणि आपल्या यंत्रणेने तो भारच फेकून दिला होता.

उपाय काय असेल, ते तुमच्या रोजच्या जगातून घ्या. Bank च्या रांगेत गर्दी झाली की व्यवस्था काय करते? **Token क्रमांक**. माणसं कशीही उभी राहोत, पटांगणात विखुरलेली असोत: प्रत्येकाच्या हातात “क्र. 47” ची चिठ्ठी: क्रम चिठ्ठीत, जागेत नाही. यंत्रात नेमकं हेच: **प्रत्येक token च्या vector मध्ये त्याच्या जागेचा शिक्का मिसळायचा**: “मी शब्द क्र. 1”, “मी क्र. 7”. आता “रामाने” (क्र. 1 चा शिक्का) आणि “रामाला” (क्र. 3 चा शिक्का) हे बाजारात **वेगळे** दिसतात: पाट्या-मागण्यांमध्ये जागेचा गंध उतरतो, आणि जुळणी क्रम **वापरू** शकते: “क्रियापदाआधीचा ‘ने’-वाला = करणारा” ही नजर आता शिकता येते. या शिक्क्याचं नाव: **positional encoding** (जागेचं encoding: Book 1, 2.2 चा जुना शब्द, नव्या कामावर).

शिक्का नेमका कसा बनतो याचे तपशील अनेक (आणि बदलते) आहेत: पण दोन कल्पना पुरे: (1) साधा “1, 2, 3” आकडा चालत नाही: हजारव्या शब्दाचा आकडा राक्षसी होऊन अर्थालाच झाकतो: म्हणून शिक्के हलक्या, लयबद्ध लहरींनी बनवतात: घड्याळाच्या काट्यांसारखे (सेकंद-काटा जलद फिरतो, तास-काटा हळू: दोन्ही मिळून वेळ नेमकी सांगतात: तसे जलद-हळू लहरींचे मिळून जागेची सही). (2) नव्या यंत्रांत शिक्का “माझी जागा” पेक्षा **”आपल्या दोघांतलं अंतर”** सांगणारा असतो (relative): कारण भाषेला “क्र. 847” पेक्षा “माझ्या मागचा तिसरा” हेच जास्त लागतं: आणि याच सुधारणेने खिडक्या लांबवणं (1-2 million tokens: 2.10) सोपं झालं.

बस. भोक बुजलं. आता यादी पूर्ण झाली आहे: अर्थाच्या जागा (3.1), संदर्भाने सरकणं (3.3), निवडक-वजनदार बघणी (3.5), अनेक नजरा (3.6), आणि क्रमाचा शिक्का (3.7). पुढच्या chapter मध्ये हे सगळे भाग एका इमारतीत चढतात: transformer.

इथे लोक काय चुकीचं समजतात

”क्रम ही इतकी मूलभूत गोष्ट: ती ‘जोडलेली चिठ्ठी’ असेल हे विचित्र वाटतं: रचनेतच का नाही?” उलटं बघा: क्रम रचनेत **होता**: जुन्या RNN मध्ये (3.4): एक-एक शब्द, क्रमानेच: आणि त्याच रचनेने पिशवी गळवली आणि GPU रिकामी ठेवली. Transformer चा सौदा जाणीवपूर्वक आहे: **क्रम रचनेतून काढून data त**

टाकला (शिवका बनवून): रचना समांतर-मोकळी झाली, क्रम शिवक्याने परत आला. Book 1, 7.1 ची आठवण येते का? “State servers मधून काढून database त टाका: servers रिकामे, वाढ मोकळी”: **नेमका तोच नमुना**: जड गोष्ट रचनेतून काढून सोबतच्या नोंदीत टाकणं. चांगल्या रचना एकाच युक्तीची रूपं पुन्हा-पुन्हा वापरतात.

MAP वर

हा छोटा शिवका थेट पैशाच्या दोन ओळींना टेकलेला आहे: (1) **लांब-context ची शर्यत**: “आमची खिडकी 1M/2M tokens” ही जाहिरात (Gemini-कुळाची खासियत) शक्य झाली ती relative-शिवक्यांच्या सुधारणांमुळे: खिडकी हा फक्त memory चा नाही, **क्रम-शिवक्याचा** प्रश्न होता; (2) “मधलं हरवणं”: खूप लांब रांगेत यंत्र सुरुवात- शेवट लक्षात ठेवतं, मध्य विसरतं (“lost in the middle” म्हणून मोजलं गेलेलं वागणं: chapter 4.5 मध्ये पूर्ण): कागदपत्रं देताना महत्वाचं **सुरुवातीला/शेवटी** ठेवा हा व्यावहारिक नियम इथून निघतो: आणि हो, या पुस्तकाच्या “महत्वाचं आधी, मग तपशील” रचनेचंही तेच कारण आहे.

स्वतः बघा (5 मिनिटं)

क्रमाचा भार स्वतः तोला: Claude/ChatGPT ला द्या: “या दोन वाक्यांत अर्थाचा फरक काय: ‘फक्त मी तुला 100 रुपये देईन’ / ‘मी तुला फक्त 100 रुपये देईन’ / ‘मी फक्त तुला 100 रुपये देईन’.” एकच शब्द (“फक्त”) तीन जागा फिरला: तीन वेगळे अर्थ: आणि यंत्र तिन्ही नेमके फोडून दाखवेल: जागेचा शिवका जुळणीत मुरलेला आहे याचा जिवंत पुरावा. (मराठी “फक्त” हा क्रम- संवेदनशीलतेचा उत्तम खेळाडू आहे: वाक्यात फिरवत राहा!)

विचार करा

1. (derivation) समजा शिवके **काढून** टाकले (जागेचा गंध शून्य): यंत्र पूर्ण निकामी होईल का? कुठली कामं तरीही जमतील, कुठली कोसळतील? (जुळणी “फक्त मजकुरावर” उरते, एवढ्यावरून तर्क करा.)

उत्तर: जिथे क्रम बिनमहत्त्वाचा तिथे यंत्र चालेल: शब्दांची पिशवी पुरते अशी कामं: विषय ओळखणं (“हा मजकूर cricket चा आहे”), ढोबळ भावना (रागीट/खूश), शब्द-मोजणी-छाप शोध. कोसळेल तिथे, जिथे अर्थ **रचनेत** आहे: कोणी-कोणाला (कर्ता-कर्म!), नकाराची जागा (“न आलेला पाऊस” vs “आलेला न-पाऊस?”), हिशोबाची पायरी-रचना, code (क्रम म्हणजेच program: Book 1!). म्हणजे शिक्का काढला की यंत्र “विषय-ओळखू” राहतं, “भाषा-जाणतं” मरतं. उलटा धडा: तुमच्या कामात अर्थ क्रमात किती आहे हे आधी तोला: “review चा सूर सांग” ला पिशवी पुरते; “करार वाच आणि कोण-कोणाला-काय-देणं ते काढ” ला क्रमाची पूर्ण इमारत लागते: आणि आता ती इमारत बघायला तुम्ही तयार आहात: पुढचा chapter.

Chapter 3.8 [SPINE]: Transformer, पूर्ण जोडून

हा या section चा कळस. हातातले भाग मोजा: अर्थाच्या जागा (3.1), संदर्भाने सरकणं (3.3), मागणी-पाटी-मालाचा बाजार (3.5), अनेक नजरा (3.6), क्रमाचे शिक्के (3.7). आज हे सगळे भाग एका इमारतीत चढवायचे: आणि त्या इमारतीचं नाव आहे **transformer**: GPT मधला T, आणि Claude-Gemini-Llama या सगळ्यांचा एकच सांगाडा.

इमारत मजल्यांची आहे; एक **मजला** (layer) आधी नीट बघू. प्रत्येक मजल्यावर दोनच खोल्या, ठरलेल्या क्रमाने:

खोली 1: बाजार (attention). मजल्यावर आलेल्या प्रत्येक token- vector ची इतर सगळ्यांशी देवाणघेवाण: 3.5 चा बाजार, 3.6 च्या अनेक नजरांसह. इथे **शब्द एकमेकांशी बोलतात**: “तो” मध्ये “मुलगा” मुरतो, “पान” विड्याच्या वस्तीकडे सरकतो.

खोली 2: एकांत (छोटं गणित-जाळं). बाजारातून आल्यावर **प्रत्येक token स्वतंत्रपणे** एका छोट्या, शिकलेल्या गणित-जाळ्यातून जातो (शेजाऱ्यांकडे न बघता): जमवलेल्या मालावर **प्रक्रिया**: मिसळलेल्या अर्थातून नवे गुण काढणं. बाजारात खरेदी, घरी येऊन स्वयंपाक: गोळा करणं आणि पचवणं, दोन वेगळ्या खोल्या. (आणि आठवणीसाठी: 2.4 ची बहुतांश वजनं याच “एकांत-खोल्यांमध्ये” राहतात.)

बस: बाजार, मग एकांत: एक मजला संपला. आता **इमारत**: असे मजले एकावर एक: छोट्या यंत्रांत 12-30, मोठ्यांत 80-100+. आणि इथे या chapter ची मधली, सगळ्यात महत्वाची कल्पना: **प्रत्येक मजल्यावर “शब्द” अधिकाधिक “विचार” होत जातो**. खालच्या मजल्यांवर जुळण्या साध्या असतात (शेजारचे शब्द, व्याकरणी जोड्या); मधल्या मजल्यांवर वाक्य-पातळी (कोण-कोणाला, कुठल्या वस्तीत); वरच्या मजल्यांवर संवाद-पातळी (सूर, हेतू, “आता उत्तरात काय यायला हवं”). कोणीही मजल्यांना कामं **वाटलेली नाहीत** (मडकं, शेवटचा अवतार!): खोल-वर ही शिडी चक्रातून **उगवते**. Book 1, 1.5 चा मनोरा आठवा: transistor -> gate -> बेरीज -> recipe: **तीच थर-रचना, इथे अर्थाची**: token -> जुळणी -> वाक्य-समज -> उत्तराची दिशा.

आता पूर्ण प्रवास, एका दमात (1.1 चं वचन पुरं करू): तुम्ही Enter दाबता -> tokens (1.6) -> प्रत्येकाला जागा-vector (3.1) + क्रम-शिक्का (3.7) -> **मजला 1: बाजार + एकांत -> मजला 2 -> ... -> मजला 90** -> सगळ्यात वरच्या मजल्यांवर शेवटच्या token चा vector आता “पुढे काय यावं” या प्रश्नाचं मूर्त रूप झालेला असतो -> त्याची token-वहीशी जुळणी -> **probability-यादी** (1.3) -> फासे (1.5) -> एक token बाहेर. आणि पुन्हा सगळं, पुढच्या token साठी. Part 1 च्या “जिलेबीच्या वेटोळ्या”मागे हे चालू होतं: प्रत्येक वेटोळ्यामागे नव्वद मजल्यांची एक चढाई.

2017 साली Google च्या आठ संशोधकांनी ही रचना एका paper मध्ये मांडली: “Attention Is All You Need” (3.4 मध्ये भेटलेलं नाव). तेव्हा ते भाषांतरासाठी होतं; कोणाला कल्पनाही नव्हती की हाच सांगाडा दशकभरात जगातली सगळ्यात मौल्यवान रचना होईल. का हीच जिंकली, हे आता तुम्ही स्वतः सांगू शकता: लांब नाती थेट (पिशवी नाही: 3.4), सगळं समांतर (GPU-फौज पूर्ण जुंपलेली), आणि रचना **एकसुरी-सोपी** (तोच मजला पुन्हा-पुन्हा: छापायला आणि मोठं करायला सोपं: पुढचा chapter याच “मोठं करण्या”चा आहे).

इथे लोक काय चुकीचं समजतात

“GPT, Claude, Gemini: यांची ‘रहस्यं’ वेगळी असणार.” सांगाडा जवळजवळ **एकच** आहे: transformer चे मजले. मग फरक कुठे? Data आणि चाळण्या (2.7-2.8), शिकवणी-चव (2.9), आकार-मजले-heads ची मापं, आणि serving च्या युक्त्या (Section 4). म्हणजे स्पर्धा “गुप्त रचनेची” नाही; **घडवण्याच्या पाककृतीची** आहे: सगळ्यांकडे तंदूर तोच; बिर्याणी वेगळी. (आणि म्हणून open-weights यंत्रं बंद-यंत्रांच्या मागोमाग इतक्या वेगाने पोहोचतात: सांगाडा लपवता येत नाही.)

MAP वर

एक paper -> दहा वर्षांत trillion-dollar उद्योग: हा या दशकातला सगळ्यात मोठा leverage-किस्सा आहे (Book 1, 0.2 चं सूत्र टोकाला): आठ पगारदार संशोधक, एक खुला paper, आणि त्यावर उभं जग. दोन धडे: (1) **खुल्या ज्ञानाची ताकद**: paper बंद ठेवला असता तर? इतिहास सांगतो: बंद खजिने कुजतात, खुले पेरले जातात (Book 1, 3.7); (2) **रचना जिंकते तेव्हा पुरवठादार जिंकतो**: transformer GPU-प्रेमी निघाला: आणि Nvidia \$5.5 trillion झाली. पुढच्या वेळेस कुठलीही नवी “जादूची रचना” बातमीत आली, की पहिला प्रश्न विचारा: **ही कुठल्या हत्याराशी जुळते, आणि ते हत्यार कोण विकतं?**

स्वतः बघा (5 मिनिटं)

मजल्यांची शिडी बाहेरून जोखा: Claude/ChatGPT ला एकच वाक्य द्या: “काय रे, आज पण उशीर?” आणि विचारा: “या वाक्यातून जे-जे कळतं त्याची यादी कर.” उत्तरात थर दिसतील: शब्द-पातळी (प्रश्न आहे), नाते-पातळी (बोलणारा ओळखीचा, ‘रे’ वरून), इतिहास-पातळी (‘पण’ वरून: उशीर पूर्वीही झालाय), सूर-पातळी (सौम्य चिडचिड/थट्टा). चार शब्दांतून चार मजले: खालून वर. हीच चढाई प्रत्येक token मागे, नव्वद मजल्यांवर, चालू असते.

विचार करा

1. (derivation) “बाजार + एकांत” ही जोडी **दोन्ही** का लागते? फक्त बाजार-बाजार-बाजार असे मजले चढवले तर काय हरवेल: आणि फक्त एकांत-एकांत ठेवलं तर? (खोल्यांची कामं आठवा.)

उत्तर: फक्त बाजार = नुसती मिसळण: प्रत्येक मजल्यावर vectors एकमेकांच्या सरासरीकडे सरकत जातात: शंभर मजल्यांनंतर सगळे शब्द एकसारखे “गुळगुळीत” (सूप!): गोळा खूप, पचन शून्य. फक्त एकांत = प्रत्येक token आपल्याच विश्वात: “पान” कधी विड्याकडे सरकणारच नाही (3.3 ची problem जशीच्या तशी परत): पचन तयार, ताट रिकामं. भाषेला दोन्ही लागतं: **जोडणी** (अर्थ शेजाऱ्यांत असतो) आणि **प्रक्रिया** (जोडलेल्यावर नवं काढावं लागतं). आणि हा नमुना ओळखीचा आहे: Book 1, 8.1 चं 60-seconds- चित्र आठवा: network ने गोळा, machine ने प्रक्रिया: गोळा-पचव, गोळा-पचव: चांगल्या रचना, पुन्हा एकदा, एकच गाणं वेगळ्या सुरात गातात.

Chapter 3.9 [SPINE]: मोठं केल्याने चांगलं का होत गेलं

Transformer 2017 चा. ChatGPT चा स्फोट 2022 चा. मधली पाच वर्ष काय चाललं होतं? उत्तर एका शब्दात: **मोठं करणं**. तीच रचना: जास्त मजले, जास्त वजनं, जास्त data, जास्त GPU. आणि या “फक्त मोठं” मधून जे घडलं, ते या क्षेत्रालाही अनपेक्षित होतं: म्हणून हा chapter: कारण इथेच AI-गुंतवणुकीच्या लाटेचं आणि तिच्या शंकांचं, दोन्हीचं मूळ आहे.

पहिली गोष्ट: **नियमितता**. संशोधकांनी मोजून बघितलं: वजनं दुप्पट, data दुप्पट, compute दुप्पट करत गेलं की चूक (loss: 2.5) कशी घटते? आणि सापडलं ते विलक्षण: घट **गुळगुळीत, अंदाज-लावता-येणाऱ्या** रेषेने होते: आलेख काढला की सरळ रेषा (नाव: **scaling laws**). याचा व्यापारी अर्थ प्रचंड: “\$500 million लावले तर यंत्र **साधारण किती** चांगलं होईल” हे **आधीच** सांगता येऊ लागलं: आणि जिथे परतावा अंदाज-योग्य, तिथे भांडवल धबधब्यासारखं येतं (1.4 चं सूत्र: शक्यता x किंमत: इथे शक्यता मोजता आली!). AI- लाटेमागचं गणित हेच.

दुसरी गोष्ट, आणि ही अजून विलक्षण: **उड्या**. चूक गुळगुळीत घटते: पण **क्षमता** गुळगुळीत वाढत नाहीत! छोटं यंत्र भाषांतर जमवत नाही-जमवत नाही... आणि एका आकारापलीकडे **अचानक** जमवायला लागतं. अंकगणित, विनोद समजणं, “step-by-step विचार”: अशा अनेक गोष्टी कुठल्याशा उंबरठ्यावर **उगवल्या** (नाव: emergence): कोणी शिकवल्या नाहीत, design केल्या नाहीत. का? प्रामाणिक उत्तर: पूर्ण कळलेलं नाही. एक चित्र मदत करतं: पाणी 99 अंशावरही पाणीच असतं; 100 व्या अंशाला **वाफ**: गुण हळू चढतो, **रूप** अचानक बदलतं. (आणि एक सावधगिरी: काही “उड्या” मोजपट्टीच्या खेळाही निघाल्या: 2.5 चा Goodhart इथेही जागा असतो.)

मग प्रश्न साहजिक: **मोठं करत राहिलं की सगळं येईल?** इथे जमिनीवरची तीन वळणं, आजची (2026):

वळण 1: अन्न संपतंय. Data ची विहीर आटते आहे (2.7): “अजून दुप्पट चांगला data” आणायचा कुठून? Synthetic, करार, वापर-data: सगळे रस्ते महाग किंवा मर्यादित.

वळण 2: बिल चढतंय. प्रत्येक पिढी ~दुप्पट-चौपट महाग: \$100M -> \$1B -> पुढे? (2.6 चे आकडे). रेषा सरळ असली तरी ती **लांबवणं** exponential महाग आहे: आणि परतावा प्रत्येक पायरीला “तितकाच” मोठा राहीलच याची हमी नाही.

वळण 3: नवी दिशा उघडली. म्हणून 2024-26 ची मोठी उडी “अजून मोठं यंत्र” पेक्षा वेगळीकडून आली: **उत्तराआधी जास्त विचार**. यंत्राला उत्तर देण्याआधी (न-दाखवले जाणारे) हजारो “विचार-tokens” खर्चू द्या: पायऱ्या मांडू द्या, स्वतःला तपासू द्या: तेच यंत्र गणित-code-नियोजनात झेप घेतं. (बाजारातली नावं: “reasoning/ thinking” models: GPT-5.6 च्या Sol-श्रेणीचं आणि Claude च्या extended-

thinking चं हेच रहस्य.) म्हणजे नवं सूत्र: **शिकताना मोठं (training scale) + उत्तर देताना वेळ (thinking time):** दुसरा काटा पहिल्यापेक्षा स्वस्त आहे: पण तो **प्रत्येक उत्तराला** लागतो: token-bill वाढतो (1.1 चं चक्र, आता लांब!).

इथे लोक काय चुकीचं समजतात

दोन टोकं, दोन्ही बाजारात जोरात: **"Scaling संपलं, AI ची भिंत आली"** आणि **"मोठं करत राहू, सगळं येईल."** दोन्हींशी सावध: पहिल्याने 2019 पासून दरवर्षी भिंत जाहीर केली आणि दरवर्षी यंत्रं झेपावली; दुसऱ्याकडे data-बिलाच्या वळणांचं उत्तर नाही. प्रामाणिक चित्र: **एक रस्ता चढा होतोय, म्हणून फाटे फुटले आहेत** (विचार-वेळ, चांगला data, छोटी-नेमकी यंत्रं, हत्यारं- जोडणी: Part 3). "एकच रेषा पुढे ओढणं" हा अंदाज-पद्धतीचा सापळा आहे: Book 1, 4.5 चा estimate-धडा इथे उद्योग-पातळीवर लागू.

MAP वर

Scaling laws नी AI ला **भांडवली शर्यत** बनवलं: अंदाज-योग्य परतावा -> महाकाय गुंतवणुका -> जे टेबलावर billions ठेवू शकतात तेच frontier-खेळाडू (मूठभर companies + मोजके देश). पण "उड्यांची" अनिश्चितता उलटी संधीही देते: उंबरठ्यापलीकडचं यंत्र **आधीच्या पिढीला अशक्य** वाटलेली कामं करतं: म्हणजे "AI ला हे जमत नाही" ही तुमची यादी दर सहा महिन्यांनी **पुन्हा तपासायची** असते (1.2 ची यादी: ती गोठवू नका!). Founder-नियम: क्षमतांची यादी कागदावर नाही, calendar वर ठेवा.

स्वतः बघा (5 मिनिटं)

"विचार-वेळ" चा फरक स्वतः मोजा: एखादं बहु-पायरी कोडं घ्या (उदा.: "माझ्याकडे 3 डबे आहेत; पहिल्यात दुसऱ्याच्या दुप्पट लाडू; तिसऱ्यात पहिल्यापेक्षा 4 कमी; एकूण 26: प्रत्येकात किती?"). आधी विचारा: "फक्त अंतिम उत्तर दे, स्पष्टीकरण नको." मग नव्या chat मध्ये: "पायरी-पायरीने सोडव." दुसऱ्या पद्धतीची अचूकता जाणवण्याइतकी चांगली असते: तुम्ही यंत्राला "विचार-tokens" खर्चायची परवानगी दिलीत (1.6 ची "उलगाडायला लावा" युक्ती हीच होती!). आणि हो: दुसरं उत्तर महागही होतं: विचार tokens मध्येच येतो.

विचार करा

1. (derivation) Scaling law सरळ रेषा दाखवते: तरी "पुढची पिढी बनवावी का" हा निर्णय आपोआप का होत नाही? **Rेषेच्या दोन्ही अक्षांकडे** बघा आणि व्यापारी पेच मांडा.

उत्तर: रेषा सांगते “इतकं compute -> इतकी कमी चूक”: पण (अ) x-अक्ष **log** आहे: प्रत्येक समान सुधारणेला **दसपट** खर्च: \$1B नंतरची पायरी \$10B ची; (ब) y-अक्ष “चूक” आहे, **”ग्राहक-मूल्य”** **नाही**: loss 2.1 वरून 1.9 झाला याचे पैसे कोण, किती देईल? उंबरठा-उडी आली तर सोनं; नाही आली तर “थोडं बरं” यंत्र दसपट किमतीत. म्हणजे पेच: खर्च नक्की-exponential, परतावा शक्यता-उडी: हे lottery-अर्थशास्त्र आहे, कारखाना- अर्थशास्त्र नाही. म्हणून frontier-शर्यत मूठभरांची: आणि म्हणूनच **वापरणाऱ्यांसाठी** सुवर्णकाळ: शर्यतीचा खर्च ते करतात, घटलेले token-भाव तुम्हाला मिळतात (GPT-5.6 Luna \$0.20/M ची 80% कपात आठवा!). शर्यत बघा, धावू नका: भाड्याने जिंका.

Chapter 3.10 [SPINE]: Photo कसं वाचतं

तुम्ही Claude/ChatGPT ला किराणा-यादीचा photo पाठवता आणि म्हणता “याची नीट यादी बनव”:
आणि यंत्र, जे आत्तापर्यंत आपण “शब्द-यंत्र” म्हणून उघडलं, ते **चित्र** वाचून दाखवतं. शब्दांची इमारत
(transformer) डोळे कुठून शिकली?

उत्तराचा पाया Book 1 मध्ये केव्हाच घातला गेला होता: **photo म्हणजे numbers** (2.2: ठिपके,
प्रत्येकाचे तीन आकडे). आणि Part 2 ने आत्ता शिकवलं: transformer ला मुळात “शब्द” लागतच नाहीत:
लागते **vectors ची रांग** (3.8: token -> जागा-vector -> मजले). मग युक्ती जवळजवळ उघडी पडली
आहे: photo ची **रांग** बनवा!

कशी: photo ला जाळी टाका: 16x16 ठिपक्यांचे छोटे **चौकोन** कापा (एक photo = काहीशे चौकोन).
प्रत्येक चौकोन = काही-शे numbers = त्याला एक जागा-vector द्या (embedding ची जुनी युक्ती: 3.1)
+ जाळीतल्या जागेचा शिक्का (3.7 चा positional, आता दोन-मितींचा!). झाली रांग: **“दृश्य-tokens”**
ची. आणि पुढे? **तीच इमारत, तेच मजले**: बाजारात चौकोन एकमेकांकडे बघतात (“हा तपकिरी चौकोन
आणि तो शेपटीवाला: एकाच मांजराचे भाग”), एकांतात प्रक्रिया: 2.2 च्या “मांजर-problem” चं उत्तर
शेवटी हेच निघालं: नियम नाहीत, चौकोनांचा बाजार.

आणि खरी झेप पुढे: शब्द-tokens आणि दृश्य-tokens **एकाच रांगेत** मिसळ्या! तुमचा photo + “याची
यादी बनव” हा प्रश्न: दोन्ही tokens ची एकच रांग, एकाच इमारतीतून: आणि attention ला आता **चित्रा-**
कडून-शब्दाकडे बघता येतं: “यादी” हा शब्द photo तल्या अक्षर-चौकोनांकडे बघतो. एकाच जागेत
सगळ्या प्रकारांची भाषा: याचं बाजारी नाव **multimodal** (बहु-रूपी): आजची सगळी मोठी यंत्रं (GPT-
5.6, Claude 5, Gemini 3) अशीच आहेत. आणि उलट दिशाही उघडली: रांगेतून चित्र-tokens
जन्माला घालता येतात: image generation: तीच जिलेबी (1.1), फक्त वेटोळी शब्दांची नाही,
चौकोनांची.

मोठं तात्त्विक वाक्य, या पुस्तकाच्या दोन्ही खंडांना जोडणारं: **Book 1 ने दाखवलं “सगळं numbers**
होऊ शकतं” (2.2); Book 2 दाखवतं “numbers च्या रांगेवर एकच इमारत पुरते.” कापणी वेगळी
(शब्दांचे tokens, चित्रांचे चौकोन, आवाजाचे तुकडे: पुढचा chapter), इमारत एक. म्हणून “AI ला डोळे
आले” ही बातमी वाचताना आता तुम्हाला जादू दिसणार नाही; **रांगेत नवा प्रकार जोडला** एवढंच दिसेल.

इथे लोक काय चुकीचं समजतात

“यंत्र photo ‘बघतं’ म्हणजे माणसासारखं दृश्य अनुभवतं.” आठवा 1.6 चा चष्मा: तिथे अक्षरं दिसत नव्हती;
इथे **ठिपक्यांचे बारकावे** चौकोनात विरघळतात. म्हणून विचित्र-वाटणाऱ्या चुका: घड्याळाचे नेमके काटे,
गर्दीतल्या सहा बोटांचा हात, बारीक अक्षरांतली एखादी जोडाक्षर-चूक: चौकोनाच्या ठोकळ्याखालचं जग

धूसर. आणि तरी “यादी वाचणं” उत्तम जमतं: कारण ते चौकोन-नमुन्यांचं काम आहे. नियम तोच जुना (1.6): **चूक random नाही; चष्याच्या आकाराची आहे**: फक्त चष्या आता चौकोनी.

MAP वर

Multimodal चं अर्थकारण Book 1, 2.1 च्या जुन्या प्रमाणावर उभं आहे: **text : photo : video = 1 : हजार : लाख**. एक photo = काहीशे-हजार tokens (pricing-पानांवर “image tokens” ची वेगळी मोजणी असते ती हीच); video = प्रति-second चा token-महापूर. म्हणून photo-प्रश्न text-प्रश्नाहून महाग, आणि video-समज अजून महाग-मर्यादित. Founder-गणित: “receipt/form/यादी photo तून वाचणं” ही आज **स्वस्त-पिकलेली** संधी आहे (काकांच्या दुकानाची वही: photo काढ -> नोंदी आपोआप!; Book 1, 7.9 चं दुखणं, नवं हत्यार); “video समजणं” अजून महाग-कच्चं: तिथे घाई नको.

स्वतः बघा (5 मिनिटं)

चौकोनी चष्या तपासा: एक photo काढा ज्यात मोठं + बारीक दोन्ही आहे (दुकानाची पाटी लांबून: मोठं नाव + खाली बारीक phone-number). यंत्राला विचारा: “पाटीवर काय लिहिलंय?” मोठं नाव बिनचूक येईल; बारीक number कधी बरोबर, कधी एखादा आकडा घडवलेला (धूसर चौकोन + प्रवाही भाषा: 2.4 चं सूत्र, दृश्य-रूपात!). मग तोच भाग जवळून, स्पष्ट photo त द्या: उत्तर सुधारेल. धडा व्यावहारिक: **यंत्राला द्यायचा photo म्हणजे यंत्राचा चष्या: स्पष्ट द्या, नेमकं मिळेल.**

विचार करा

- (derivation) शब्दांची रांग एका-दिशेची (डावी-उजवी); photo चौकोनांची जाळी **दोन-मिती**. Attention च्या बाजाराला (3.5) याने फरक पडतो का: की काहीच नाही? आणि क्रम-शिकव्याचं (3.7) काय?

उत्तर: बाजाराला फरक **पडत नाही**: जुळणी “प्रत्येकाची प्रत्येकाशी” आहे: रांग सरळ असो की जाळी, पटांगण एकच (हीच transformer ची लवचीकता: म्हणून एकच इमारत सगळ्या प्रकारांना). फरक पडतो **शिकव्याला**: “क्र. 47” पुरत नाही; चौकोनाला (ओळ, स्तंभ) असा दोन-आकडी गंध लागतो: नाहीतर “वर-खाली/डावी- उजवी” ही दृश्य-नाती हरवतील (3.7 चा धडा: क्रम रचनेतून काढला, मग शिकव्यात भरावाच लागतो: मिती वाढल्या, शिकका वाढला). खोल नमुना पुन्हा: **इमारत सामान्य ठेवा; प्रकाराचं वेगळेपण दारावरच्या रूपांतरात (कापणी + शिकका) ठेवा**. Book 1, 7.4 च्या API-design चीच ही आठवण आहे: गाभा एक, adapters वेगळे.

Chapter 3.11 [DEPTH]: आवाज आणि video

(DEPTH chapter, section चा शेवटचा, आणि मुद्दाम छोटा: 3.10 ची युक्ती अजून दोन प्रकारांवर फिरवायची: नवं तत्त्व शून्य, नवे परिणाम दोन-तीन.)

आवाज. Book 1, 2.2 ने कापणी तेव्हाच शिकवली होती: आवाज = कंपनाचे नमुने, ~44,000 आकडे/second. हे थेट रांगेत घालणं म्हणजे token-पूर: म्हणून आधी आवाजाचे छोटे **काळ-तुकडे** (काही-काही millisecond चे), प्रत्येकाचा एक vector (त्या क्षणाचा “आवाजी रंग”: कुठल्या पट्ट्या किती: घुमारा, कंप, s-चा शिसारा), + काळाचा शिक्का: झाली **ध्वनी-tokens ची रांग**: तीच इमारत. यातूनच: बोलणं-ते-मजकूर (voice typing: तुमच्या keyboard चा mic!), मजकूर-ते-बोलणं (उलटी जिलेबी: आवाज-tokens जन्मतात), आणि आता **थेट आवाज-ते-आवाज** संवाद: मध्ये मजकुराचा टप्पा न घेता: म्हणून नव्या voice-assistants ना तुमचा **सूरही** कळतो (घाई, चिडचिड, थट्टा): कारण सूर मजकुरात नव्हता, **ध्वनी-tokens मध्ये** आहे. (जुनी साखळी: आवाज->मजकूर->विचार->मजकूर->आवाज: सूर पहिल्याच टप्प्यावर गळायचा: आता गळत नाही.)

Video. = photo-चौकोन (3.10) x प्रत्येक frame + काळाचा तिसरा शिक्का (ओळ, स्तंभ, **क्षण**). तत्त्व संपलं: आता फक्त मोजणी बघा, कारण मोजणीच राजा आहे: 1 minute चा video = हजारो frames x शेकडो चौकोन = **लाखो tokens**: आणि attention चा n-squared राक्षस (3.4) इथे खरा पेटतो. म्हणून आजची यंत्रं युक्त्या करतात: frames गाळणं (प्रत्येक नको: शेजारचे जवळजवळ सारखेच: Book 1, 2.2 च्या compression-धड्याचं दृश्य-रूप), हालचाल-नमुने एकत्र दाबणं. तरीही: video-समज हे आजचं सगळ्यात **महाग** काम, आणि video-निर्मिती (Sora-कुळ: मजकुरातून चलचित्र) सगळ्यात token-खादाड: म्हणूनच ती सेवा सगळ्यात आधी मर्यादा-रांगांमध्ये (Book 1, 7.6 चं दार-आवळणं) दिसते.

आणि एक वाक्य पुढच्या दारासाठी: कान-डोळे आले: आता **हात** येणं बाकी: यंत्राने फक्त वर्णन करणं नाही, **कृती** करणं: फाईल बदलणं, ticket काढणं, code चालवणं. तो Part 3 चा पडदा आहे (agents), आणि तिथेच हे पुस्तक तुमच्या रोजच्या Claude Code पर्यंत पोहोचतं.

इथे लोक काय चुकीचं समजतात

”आवाज-कळणं म्हणजे यंत्र ‘ऐकतं’, video-कळणं म्हणजे ‘बघत बसतं’.” नमुना आता पाठ आहे: ऐकणं = ध्वनी-tokens ची रांग-प्रक्रिया; बघणं = चौकोन-रांग. आणि म्हणून चुकाही अंदाज-योग्य: गोंगाटातलं बोलणं (tokens गढूळ), दोघांचं एकदम बोलणं, video तली **नेमकी क्षण-मोजणी** (“चेंडू नेमका कितव्या second ला सुटला?”: frames गाळलेले असतील तर तो क्षण **रांगेत नाहीच**). चष्मा कळला की मर्यादा कळते: तिसऱ्यांदा, तोच नियम.

MAP वर

आवाजाचं अर्थकारण भारतासाठी खास आहे: देशाचा मोठा हिस्सा typing पेक्षा **बोलणं** पसंत करतो: voice-first products (शेतकरी-सल्ला, आरोग्य-प्रश्न, दुकानदाराची नोंद: “आज पन्नास किलो साखर संपली” **बोलून** वहीत!) ही इथली उघडी जागा आहे: आणि 2.7 ची आठवण: मराठी-बोली ध्वनी-data जगाच्या पोल्यात जवळजवळ नाही: जो गोळा करेल त्याची खाण. Video-निर्मितीचं अर्थकारण उलटं: token-खादाड = GPU-खादाड = फक्त मोठ्यांचा खेळ: तिथे **वापरणारे** व्हा (जाहिरात-clip, product-video: स्वस्त होत जाणार), **बांधणारे** नको.

स्वतः बघा (5 मिनिटं)

सूर-tokens चा पुरावा: phone च्या voice-assistent शी (किंवा voice-mode वाल्या ChatGPT/Gemini शी) एकच वाक्य दोन सुरांत बोला: “वा, छानच झालं” (खरं कौतुक) आणि तेच उपरोधाने. जुनी यंत्रं दोन्हींना एकच मजकूर देतील; नवी ध्वनी-थेट यंत्रं फरक पकडू लागली आहेत. जे पकडलं-न-पकडलं ते नोंदवा: तुम्ही आता “साखळी कुठे तुटते” ची चाचणी घेतलीत: engineer ची नजर, तुमच्या हातात.

विचार करा

1. (derivation) “थेट आवाज-ते-आवाज” यंत्रात एक **धोका** जुन्या साखळी-पद्धतीत नव्हता तो आहे. सूर कळतो हे वरदान; उलटी बाजू काय? (यंत्र आता काय **जन्माला** घालू शकतं यावरून विचार करा.)

उत्तर: जे कळतं ते घडवताही येतं (जिलेबीचा नियम): सूर- tokens जन्मतात म्हणजे **आवाजाची हुबेहूब नक्कल** जन्मते: कोणाचाही आवाज, कुठल्याही सुरात: आणि इथून “आईचा आवाज काढून पैसे मागणारा phone-scam” हे आजचं खरं दुखणं (भारतात रोज घडणारं). जुन्या साखळीत आवाज मजकुरात मरायचा: नक्कल-यंत्रणा वेगळी लागायची; आता एकाच यंत्रात. म्हणून घरात **परवलीचा शब्द** ठरवा (आवाजावर विश्वास नाही: शब्दावर), आणि “आवाज = ओळख” मानणाऱ्या जुन्या systems (bank च्या voice-verification!) आता कमजोर झाल्या हे नोंदवा. तंत्राचा जुना नियम, Book 1, 1.6 पासूनचा: **प्रत्येक नवी ताकद दोन्ही हातांत जाते**. पूर्ण हिशोब: Part 3.

SECTION 4: File पासून उत्तरापयंत

पाच chapters. Section 3 ने इमारत दाखवली; आता ती इमारत रोज, कोट्यवधी लोकांसाठी, पैसे मोजून कशी चालते: Book 1 चं serving- शास्त्र (Part 5) आणि Book 2 चं यंत्र यांचं इथे लग्न आहे.

हे तुमच्या business decision मध्ये कुठे येईल: तुमच्या AI- product च्या तीन रोजच्या तक्रारी: “पहिलं उत्तर यायला वेळ लागतो,” “काल वेगळं उत्तर दिलं होतं,” “मागचं विसरलं”: तिन्हींची यांत्रिक कारणं याच section मध्ये आहेत: आणि कारण कळलं की तक्रारीचं रूपांतर design-निर्णयात होतं: कुठे cache, कुठे temperature शून्य, कुठे सारांश. शिवाय API-bill मधल्या दोन गूढ ओळी (prompt-tokens वेगळे का? cache-सूट कसली?): इथेच उघडतात.

Chapter 4.1 [SPINE]: एक file उत्तर कशी देते

Part 1 च्या शेवटी model ची ओळख “गोठवलेली file” अशी झाली (2.10): 140 GB चे निव्वळ numbers, disk वर पडलेले. आणि Book 1 ने शिकवलंय: file **काहीच करत नाही**; करते ती चालणारी process (Book 1, 3.6: झोपलेली recipe vs जिवंत copy). मग तुमच्या प्रश्नापासून उत्तरा- पर्यंतचा **कारखाना** नेमका कसा उभा आहे? आजचा chapter हा पूर्ण कारखाना एका फेरीत: आणि गंमत अशी की त्यातला प्रत्येक विभाग तुम्ही **आधीच** शिकला आहात: Book 1 मध्ये.

फेरी सुरू: तुम्ही “Send” दाबता:

विभाग 1: दार. तुमचा प्रश्न https ने (Book 1, 5.6) company च्या दारात: तिथे load balancer (Book 1, 7.1!): कोट्यवधी प्रश्नांची वाटणी. आणि लगेच एक **रांग** (Book 1, 6.5): कारण पुढचा विभाग महाग आहे आणि त्याला रिकामं बसू द्यायचं नाही.

विभाग 2: भरलेली तोफ. खरा कारखाना: **GPU-servers ज्यांच्या जलद-memory त model आधीच चढवलेलं आहे.** हे वाक्य नीट वाचा: 140 GB file **प्रत्येक प्रश्नाला** disk वरून उचलणं म्हणजे मिनिटं (Book 1, 2.5: गोदाम हळू!): म्हणून file **एकदाच** चढवतात आणि servers **चोवीस तास भरलेले** ठेवतात: भट्टी पेटलेली ठेवणाऱ्या भटारखान्यासारखं: order आली की तवा गरम असतो. (आणि 140 GB च्या वर मोठी यंत्रं एका GPU त मावतही नाहीत: मग एक model **अनेक GPU वर कापून** मांडलेलं: Book 1, 7.1 चं आडवं-वाढणं, आता एका मेंदूच्या आत.)

विभाग 3: चक्की. तुमचे tokens आत: नव्वद मजल्यांची चढाई (3.8): एक token बाहेर: पुन्हा: जिलेबी (1.1). इथे एक सुंदर युक्ती: एका GPU-फेरीत **एकाच प्रश्नाची** चक्की फिरवणं म्हणजे फौज अर्धी रिकामी: म्हणून **अनेकांचे प्रश्न एकत्र दळले जातात** (batching: Book 1, 7.6 च्या bottleneck-शहाणपणाचं रूप): तुमचं उत्तर आणि जगातल्या आणखी विसांची उत्तरं: एकाच फेरीत, शेजारी- शेजारी. (म्हणून गर्दीच्या वेळेस पहिला शब्द जरा उशिरा: तुम्ही थाळीची वाट नाही, **वाफेच्या पंगतीची** वाट बघत असता.)

विभाग 4: परतीचा रस्ता. जन्मलेला प्रत्येक token लगेच तुमच्या- कडे वाहतो (streaming: म्हणून टपकणं दिसतं): https ने परत.

आणि आता दोन प्रश्न जे हा कारखाना **धंदा** बनवतात:

”का cloud च? माझ्या laptop वर का नाही?” छोटी यंत्रं (काही billion वजनं) आता खरंच phone-laptop वर चालतात (खुली-वजनं: 2.10!): पण मोठ्या यंत्राला लागतं: 100+ GB जलद memory, कापून-मांडणी, चोवीस-तास भट्टी: म्हणजे Book 1, 7.5 चं उत्तर: **गरज वर-खाली, यंत्रणा राक्षसी: भाड्याने घ्या.** तुमचा प्रश्न गेला की GPU दुसऱ्याच्या प्रश्नाला: भट्टीची ऊब वाटली जाते: cloud चं मूळ गणित, इथे टोकाला.

”एका उत्तराची खरी किंमत काय?” GPU-server चं तास-भाडं / तासाला दळले जाणारे tokens = प्रति-token खर्च: आणि त्यावर company चा हिस्सा = तुमचे API-भाव (\$5-30/million: 1.1). म्हणजे **token-भाव हे शेवटी GPU-भाड्याचं किरकोळ-रूप आहे:** आणि भाव घसरतात ते तीन कारणांनी: स्वस्त GPU-पिढ्या, चतुर दळण (batching, पुढच्या chapter चा cache), छोटी-नेमकी यंत्रं. Part 1 त हे भाव “दिलेले” होते; आता ते **मोजलेले** आहेत.

इथे लोक काय चुकीचं समजतात

”माझा प्रश्न ‘एका AI ला’ जातो.” प्रत्यक्षात: तुमचा प्रश्न एका **कारखान्याला** जातो: हजारो GPU-servers, त्यांवर एकाच गोठलेल्या file च्या शेकडो प्रती (Book 1, 6.7 चं replication: इथे “जिवंत प्रत” धंद्याचा पायाच), दारावर वाटप, आत पंगत-दळण. “AI शी बोलणं” हे भावनेने खरं; यंत्रणेने तुम्ही एका **serve-होणाऱ्या file शी** बोलता: आणि हीच फोड “AI ला माझं बोलणं आठवतं का, तो शिकतो का” या प्रश्नांची उत्तरं देते (नाही आणि नाही: 2.10: आठवण रांगेत, शिकणं बंद): file तीच, फक्त तुमची रांग वेगळी.

MAP वर

हा कारखाना म्हणजे आजची सगळ्यात मोठी पायाभूत-शर्यत आहे: inference (दळण) चा एकूण खर्च आता training पेक्षा मोठा झाला आहे: कारण training एकदा, दळण **रोज-कोट्यवधी-वेळा** (ChatGPT चे ~900M साप्ताहिक वापरकर्ते!). म्हणून बातम्यांमधल्या त्या राक्षसी घोषणा: शेकडो billions चे data-center करार, वीज-प्रकल्प, GPU- आरक्षण: हे सगळं **भट्टी पेटलेली ठेवण्याचं** भांडवल आहे. Book 1, 5.7 चा नियम शेवटचा फिरतो: सेवा-companies रस्ते-भट्ट्या विकत घेतायत: म्हणजे खरी सत्ता तिथेच आहे. तुमच्या नकाशावर: Level 4 च्या तळाशी Level 3 चा सगळ्यात जड मजला बांधला जातोय: आणि जकात (token-भाव) त्यावरून वाहते.

स्वतः बघा (5 मिनिटं)

कारखान्याची गर्दी-लय पकडा: एकच छोटा प्रश्न (“2+2?”) दिवसातून तीनदा विचारा: सकाळ, दुपार, रात्र: आणि **पहिला शब्द** यायचा वेळ अंदाजे मोजा (मनात एकवीस-बावीस मोजा). फरक जाणवेल: प्रश्न तोच, चक्की तीच: **पंगत** वेगळी. Book 1, 7.6 ची गर्दी-नजर आता AI-सेवेवर: आणि पुढच्या chapter मध्ये हा

“पहिल्या शब्दाचा वेळ” सूक्ष्मदर्शकाखाली येणार आहे.

विचार करा

1. (derivation) Batching (पंगत-दळण) company साठी सोन्याचं: GPU भरलेला. पण **तुमच्यासाठी** त्यात एक तोटा आणि एक छुपा फायदा आहे: दोन्ही काढा. (भटारखान्याची पंगत-उपमा ताणा.)

उत्तर: तोटा: **वाट**. पंगत भरेपर्यंत तुमची थाळी थांबते: गर्दीच्या वेळेस पहिला घास उशिरा (आणि निर्जन वेळेसही, थोडी वाट: पंगत जमायला हवी ना). छुपा फायदा: **भाव**. पंगतीमुळेच प्रति-थाळी खर्च कोसळतो: एकट्यासाठी भट्टी पेटवली असती तर तीच थाळी दसपट महाग: batching नसतं तर token-भाव आजचे नसते. आणि म्हणून बाजारात **दोन्ही** विकलं जातं: स्वस्त पंगत-सेवा (batch API: 50% सूट: Part 1 ची ओळ आठवा: आता कळली!) आणि महाग खाजगी-भट्टी (dedicated capacity: ज्यांना वाट परवडत नाही त्यांच्यासाठी). Book 1, 6.5 चा नियम पूर्ण वर्तुळ फिरला: “आत्ता हवं” सगळ्यात महाग असतं: AI मध्येही.

Chapter 4.2 [DEPTH]: एका request च्या आत काय होतं

(DEPTH chapter. मागच्या कारखान्यात एका प्रश्नाचा microscope- प्रवास: आणि याच्या शेवटी API-bill मधली “cache-सूट” ही गूढ ओळ पूर्ण उघडते: म्हणजे हा chapter थेट पैसे वाचवणारा आहे.)

तुमची request आत गेली: समजा 2,000 tokens चा मजकूर (तुमच्या टिफिन-app चे नियम + ग्राहकाची तक्रार) + प्रश्न. आतमध्ये काम **दोन टप्प्यांत** होतं, आणि दोघांचे स्वभाव उलटे आहेत:

टप्पा 1: वाचन (prefill). उत्तराचा पहिला token जन्मल्याआधी यंत्राला तुमचे 2,000 tokens **पचवावे** लागतात: प्रत्येकाचा बाजार-प्रवास, नव्वद मजले. पण आठवा 3.4: attention **समांतर** आहे: सगळ्या 2,000 tokens चे सगळे मजले **एकदम** दळता येतात: GPU-फौज पूर्ण जुंपलेली: राक्षसी काम, पण झपाट्याने. वाचन-टप्पा म्हणजे धरणात पाणी भरणं: मोठं, पण एकदाच.

टप्पा 2: जन्म (decode). आता जिलेबी: एक token -> रांगेत जोडा -> पुढचा. आणि इथे समांतरता **मेली**: token 51 जन्मल्याशिवाय 52 चा प्रश्नच नाही (1.1 चं चक्र: मुळातच क्रमिक: 3.4 च्या RNN-दुखण्याची आठवण: transformer ने **वाचन** समांतर केलं; **जन्म** क्रमिकच राहिला). प्रत्येक थेंब वेगळा: म्हणून output हळू, आणि म्हणून output महाग (input च्या 5-6 पट: Part 1 चं कोडं, आता यंत्रणेसह).

आता या chapter ची खरी युक्ती. जन्म-टप्प्यात प्रत्येक नवा token मागच्या **सगळ्यांकडे** बघतो (attention). मग token 500 जन्मताना मागच्या 2,499 च्या पाट्या-माल (K-V: 3.5) **पुन्हा** मोजायचे? वेडेपणा: ते बदललेलेच नाहीत! म्हणून कारखाना ते **साठवून** ठेवतो: **KV-cache** (3.5 च्या MAP मध्ये चाहूल लागलेली): एकदा मोजलेल्या पाट्या GPU-memory त तयार: नवा token फक्त **स्वतःचं** मोजतो आणि तयार पाट्यांशी जुळवतो. Book 1, 6.6 चं तत्त्व शब्दशः: महाग उत्तर जवळ ठेवा.

आणि आता पैशाची ओळ: हीच युक्ती **requests च्या पलीकडे** ताणली आहे. तुमच्या टिफिन-app चा प्रत्येक ग्राहक-प्रश्न त्याच 1,800 tokens च्या नियम-मजकुराने सुरू होतो: मग त्याचं वाचन-दळण **प्रत्येक वेळेस** का? पहिल्या request ला मोजा, KV साठवा: पुढच्या request ला (सुरुवात जशीच्या तशी असेल तर) **तयार धरण वापरा**: फक्त नवे tokens दळा. API-pricing मधली ओळ आता वाचा: “cache hit: input च्या 10% किंमत” (Part 1 ची गूढ ओळ!): म्हणजे **न-बदलणारा मजकूर पुढे ठेवा, बदलणारा शेवटी**: हा एक क्रम-बदल तुमचं bill 5-10 पट पाडू शकतो. (आणि “5-minute/1-hour cache” चे दर: धरण किती वेळ राखायचं त्याचं भाडं: Book 1, 6.6 चा शिळेपणा-सौदा, इथे पैशात छापलेला.)

इथे लोक काय चुकीचं समजतात

"AI 'विचार करत' असतो म्हणून हळू." आता फोड आहे: **वाचन झपाट्याचं, जन्म थेंबाथेंबाचा**: हळूपणा "विचारांचा" नाही, क्रमिक जिलेबीचा आहे. (आणि 3.9 ची thinking-यंत्रं मुद्दाम **जास्त** जन्म-tokens खर्चतात: तो हळूपणा निवडलेला आहे, quality साठी: पण यंत्रणा हीच.) दुसरी गल्लत: "cache म्हणजे यंत्र माझी उत्तरं लक्षात ठेवतं." नाही: cache **उत्तरं** नाही, **पाट्या-माल** (मधला हिशोब) ठेवतो: आणि तोही जुळणाऱ्या सुरुवातीपुरता. आठवण-भास आणि हिशोब-बचत या वेगळ्या गोष्टी: 2.10 ची शिस्त इथेही.

MAP वर

KV-cache हा आजच्या AI-serving-अर्थकारणाचा सगळ्यात मोठा गुणक आहे: system-सूचना (product चे नियम) हजारो-tokens च्या असतात आणि **कोट्यवधी वेळा** पुन्हा येतात: cache शिवाय हे परवडलंच नसतं. आणि design-जगात याने एक नवी शिस्त आणली: **prompt ची रचना = खर्चाची रचना**: स्थिर भाग वर (cache-योग्य), चंचल भाग खाली. Founder-चौकट: तुमच्या AI-feature चं bill वाचताना तीन प्रश्न: input किती (वाचन), output किती (जन्म: महागा!), cache- hit किती टक्के? या तीन आकड्यांत पूर्ण खर्च-गोष्ट असते: Book 1, 7.8 च्या तीन-ओळींची AI-आवृत्ती.

स्वतः बघा (5 मिनिटं)

दोन टप्पे उघड्या डोळ्यांनी: Claude/ChatGPT ला एक **लांब** मजकूर द्या (जुनं एखादं पान copy-paste: 500+ शब्द) आणि म्हणा: "एका वाक्यात सार." आता निरखा: Send दाबल्यावर एक **शांत क्षण** (वाचन-धरण भरतंय: लांब मजकूर = लांब शांतता), मग उत्तराचे शब्द **झपाझप** (जन्म छोटा: एकच वाक्य!). आता उलट करा: छोटा प्रश्न, लांब उत्तर मागा ("पावसावर 300 शब्द"): शांतता जेमतेम, टपकणं लांबलचक. दोन स्वभाव, तुमच्या घड्याळावर: पुढच्या chapter चं दार उघडलं.

विचार करा

1. (derivation) KV-cache "फुकट वेग" वाटतो: पण या पुस्तकाची सवय विचारते: **किंमत कुठे लपली?** (Book 1, 2.5 चा RAM-सौदा आणि 6.6 चा शिळेपणा: दोन्ही चष्मे लावा.)

उत्तर: दोन किमती: (1) **जागा.** पाट्या-माल GPU च्या सगळ्यात महाग memory त बसतो, आणि तो रांगेच्या लांबीसोबत वाढतो: लाख-token संवादाचा KV-cache म्हणजे कित्येक GB **प्रति-ग्राहक:** म्हणून लांब-context महाग (3.4 चा राक्षस इथे memory-रूपात), आणि म्हणून “context संपला” च्या मर्यादा (4.5 ची चाहूल). (2) **शिळेपणाची शिस्त.** Cache फक्त **जशाच्या-तशा** सुरुवातीला लागू: नियम-मजकुरात एक स्वल्पविराम बदलला की धरण फुटलं: पूर्ण पुनर्दळण. म्हणजे “स्थिर भाग खरंच स्थिर ठेवा” ही **रचना-शिस्त** पाळावी लागते: फुकट वेग नाही; शिस्तीच्या बदल्यात वेग आहे. Book 1 चं जुनं गाणं: प्रत्येक युक्ती एक सौदा: cache ही सुद्धा.

Chapter 4.3 [DEPTH]: पहिला शब्द हळू का असतो

(DEPTH chapter, छोटा: मागच्या chapter चे दोन टप्पे आता घड्याळाच्या काट्यावर: आणि “वेगवान AI” विकत घेताना नेमकं काय मोजायचं, ते हातात.)

Book 1, 7.7 आठवा: तिथे एका second चा पंचनामा केला होता (कुठे गेला वेळ?). तेच आता AI-उत्तरावर. वापरकर्त्याला जाणवणारा वेग **दोन** वेगळ्या आकड्यांत मोजला जातो, आणि दोघांची कारणं वेगळी:

आकडा 1: पहिल्या शब्दाची वाट (बाजारभाषेत: time to first token, **TTFT**). यात काय-काय साचतं: रांग+पंगतीची वाट (4.1 ची batching: गर्दीवर अवलंबून) + **वाचन-धरण** (4.2 चा prefill: तुमच्या prompt च्या लांबीच्या प्रमाणात!) + जाळ्याचा फेरा (Book 1, 5.3 ची latency: server कुठल्या देशात?). म्हणजे लांब कागदपत्रं देताच पहिला शब्द **आपोआप** उशिरा: बिघाड नाही, वाचनाचा वेळ आहे.

आकडा 2: टपकण्याची लय (tokens per second). एकदा जन्म सुरू झाला की थेंब किती वेगाने: हे यंत्राच्या आकारावर (मोठं यंत्र = जड मजले = हळू थेंब), GPU-पिढीवर, आणि दळण-गर्दीवर.

आणि दोघांची भांडणं वेगळी म्हणून **उपायही** वेगळे: TTFT कमी करायचा: prompt छोटा/cache-योग्य ठेवा (4.2!), जवळचा server निवडा, गर्दी-वेळ टाळा (batch-सूट उलट दिशेने: वाट चालत असेल तर 50% स्वस्त!). लय वाढवायची: छोटं/चपळ यंत्र निवडा (2.10 ची “कामाला पुरेसं” कसोटी: GPT-5.6 Luna छाप चपळ-श्रेणी याचसाठी आहेत).

पण या chapter चा खरा धडा तिसरा आहे, आणि तो मानसशास्त्राचा: **streaming चं शहाणपण**. समजा पूर्ण उत्तर तयार होईपर्यंत थांबवलं आणि मग एकदम दाखवलं: 20 seconds ची कोरी शांतता: वापरकर्ता गेला. त्याऐवजी थेंब **दिसू** द्या: तीच 20 seconds: पण “काम चालू आहे” ची जिवंत भावना: वाट सुसह्य. Book 1, 7.7 चं “वाटणं” (perceived speed) इथे AI-design चा पाया झाला आहे: म्हणूनच प्रत्येक chatbot टपकतो: तांत्रिक गरज (जन्म क्रमिक आहेच) आणि मानसिक युक्ती (वाट दिसते) यांचा दुहेरी विजय. आणि thinking-यंत्रांची (3.9) नवी रीत बघा: विचार-काळात “विचार करतोय...” ची पावलं दाखवणं: शांततेचंही streaming!

इथे लोक काय चुकीचं समजतात

“AI हळू आहे” ही एक तक्रार वाटते; प्रत्यक्षात **तीन** वेगळ्या तक्रारी असतात: (अ) पहिला शब्द उशिरा (TTFT: prompt/गर्दी/अंतर), (ब) टपकणं संथ (यंत्र-आकार/GPU), (क) उत्तर **लांबलचक** म्हणून एकूण वेळ (हे तर design: Part 1 चं “थोडक्यात सांग!”). न फोडता “वेगवान करा” म्हणणं म्हणजे Book 1, 7.7

चा जुना घोटाळा: चुकीच्या जागी मोठा server. आधी मोजा: कुठला आकडा दुखतोय: मग उपाय.

MAP वर

वेग आता pricing-पानांवरची **वेगळी वस्तू** आहे: तीच बुद्धी, जलद हवी तर जास्त भाव (priority/dedicated tiers), वाट चालत असेल तर निम्मा (batch). म्हणजे विक्रेते वेळ **तिन्ही रूपांत** विकतात: आत्ता-महाग, पंगत-स्वस्त, रात्र-सगळ्यात-स्वस्त: Book 1, 6.5 चं “आत्ता vs नंतर” इथे थेट rate-card झालं आहे. Founder- निर्णय: तुमच्या feature ला खरंच “आत्ता” लागतं का? ग्राहका- समोरचं उत्तर: हो (TTFT राजा); रात्रीचा report/सारांश: batch मध्ये लोटा: तेच काम, अर्ध्या किमतीत. Bill मधली सगळ्यात सोपी बचत बहुधा हीच असते.

स्वतः बघा (5 मिनिटं)

तीन आकडे वेगळे करून बघा: (1) TTFT: लांब मजकूर देऊन “एक शब्द उत्तर दे” म्हणा: शांतता मोजा (वाचन!). (2) लय: “1 ते 50 आकडे लिही” म्हणा: टपकण्याची गती निरखा. (3) एकूण-वेळ: “पावसावर निबंध” (लांब जन्म). आता तिन्ही अनुभव वेगळे ओळखता येतायत ना? मग तुम्ही AI चा वेग “मोजायला” शिकलात: आणि विक्रेत्याच्या “आमचं यंत्र वेगवान आहे” ला आता विचाराल: **कुठला आकडा?**

विचार करा

1. (derivation) Voice-संवादात (3.11 ची आवाज-यंत्रं) TTFT अचानक **राजा** का बनतो: chat पेक्षा कितीतरी कडक? आणि यातून voice-product design चा एक नियम काढा.

उत्तर: बोलण्यात शांततेची सहनशीलता वेगळी असते: chat मध्ये 3 seconds ची वाट “ठीक”; बोलताना 3 seconds ची शांतता = “phone कटला का?” (माणसांच्या संवादात प्रतिसाद-अंतर साधारण अर्ध्या second च्या आत असतं!). म्हणून voice-यंत्रणा TTFT साठी वेडीपिशी असते: छोटं-चपळ यंत्र आधी बोलायला लागतं (“हं, बघतो हं...”) आणि मागे मोठं यंत्र खरं उत्तर दळतं: दोन-यंत्री जोडी! Design-नियम: **संवाद-माध्यम जितकं “जिवंत”, तितका TTFT चा भाव चढा:** chat < voice < (उद्याचे) प्रत्यक्ष- हालचाल करणारे agents. आणि हो: “हं, बघतो हं” ही युक्ती तुम्ही रोज वापरता: दुकानदार गिन्हाईकाला “बसा, आलोच” म्हणतो: TTFT व्यवस्थापन, माणसाचं.

Chapter 4.4 [SPINE]: तोच प्रश्न, वेगळं उत्तर का

तुमच्या हाताखालच्या माणसाने सोमवारी एक उत्तर दिलं आणि मंगळवारी त्याच प्रश्नाला दुसरं: तुम्ही त्याला बेभरवशाचा म्हणाल. AI नेमकं हेच करतं: काल “हे तीन पर्याय” म्हणालं, आज “हे चार.” Book 1 च्या recipe-जगात (1.3: तेच input, तेच output: नेहमी) हे **घोटाळ्याचं** लक्षण ठरलं असतं. इथे? इथे ते **रचनेत** आहे: आणि कुठे-कुठून येतं ते फोडलं की “कधी चालवून घ्यायचं, कधी बंद करायचं” हे तुमच्या हातात येतं.

उगम 1, मुख्य आणि मुद्दाम: फासे. Part 1 चं चक्र आठवा (1.5): यंत्र probability-यादी बनवतं आणि **फासे टाकून** निवडतं (temperature). “उद्या पाऊस ____” ला “पडेल” 0.61 आणि “आहे” 0.19: दोन्ही फेऱ्यांत दोन्ही येऊ शकतात: आणि एक token वेगळा निघाला की पुढची **पूर्ण वाट** वेगळी (प्रत्येक token मागच्या रांगेवर उभा: चक्रच तसं आहे). फासे बंद करता येतात: temperature = 0: दरवेळेस यादीतला राजाच. मग सगळेच 0 का नाही ठेवत? कारण 0 ची किंमत आहे: (अ) भाषा **सपक** होते (1.5 चं मेलेलेपण): सर्जनशील कामं मरतात; (ब) यंत्र **एकाच चुकीला** चिकटतं: फासे असते तर दुसऱ्या फेरीत निसटलं असतं; (क) “दहा पर्याय दे” छाप कामांना विविधताच हवी असते.

उगम 2: रांग तीच नसते. “तोच प्रश्न” खरंच तोच होता का? Chat-apps मध्ये मागचा संवाद, तारीख, तुमच्या आठवणी-नोंदी: सगळं रांगेत जातं (2.10: आठवण रांगेत!). काल आणि आजची रांग वेगळी: मग जुळणी वेगळी (3.3: संदर्भ अर्थ सरकवतो): उत्तर वेगळं: फाशांचा दोषच नाही. (आणि app मागचं model-रूपही बदललेलं असू शकतं: 2.10 ची “engine बदलली, दुकान तेच” गोष्ट.)

उगम 3, बारीक पण खरा: कारखान्याची धूळ. Temperature 0 ठेवूनही **पूर्ण** हमी नसते: GPU वर कोट्यवधी बेरजा वेगवेगळ्या क्रमाने जुळतात (पंगत-दळण: 4.1!), आणि अति-बारीक अपूर्णाकांत क्रम बदलला की शेवटचा आकडा केसाने हलू शकतो: दोन tokens ची probability जवळजवळ बरोबरीत असेल तर राजा बदलतो. दुर्मिळ, पण “आम्ही 0 ठेवूनही एकदा वेगळं आलं” चं हे उत्तर आहे: Book 1, 4.4 चा माहोल-धडा, अपूर्णाकांच्या पातळीवर.

आता निर्णय-चौकट, founder साठी: प्रश्न “बेभरवशाचं का?” नाही; “**या कामाला विविधता ही feature आहे की bug?**” नावं-कल्पना- मसुदे: feature (फासे उघडा). हिशोब-नियम-वर्गीकरण (“ही तक्रार refund-पात्र?”): bug (temperature 0, आणि उत्तराचा **ढाचा** बांधून: “फक्त हो/नाही + नियम-क्रमांक”). आणि जिथे 0 ठेवूनही चालत नाही: तिथे Book 1, 4.2 ची जुनी शिस्त: **एकाच फेरीवर विश्वास नाही; तपासणी-फेरी वेगळी** (त्याच यंत्राला “हे उत्तर नियमांशी तपास” म्हणणं: Part 3 चं मोठं हत्यार).

इथे लोक काय चुकीचं समजतात

”वेगळं उत्तर आलं = यंत्र अविश्वसनीय = वापरण्यालायक नाही.” माणसाची मोजपट्टी पुन्हा आडवी येते (1.6 चा धडा): माणसाकडून शब्दशः-सारखं उत्तर आपण **मागतच** नाही; गाभा सारखा असला की पुरतं. यंत्राचंही तसंच मोजा: **गाभा-स्थिरता** (उत्तराचा निष्कर्ष तोच का?) वेगळी, **शब्द-स्थिरता** वेगळी. दहा फेऱ्यांत निष्कर्ष दहादा तोच पण शब्द वेगळे: उत्तम यंत्र. निष्कर्षच हिंदकळतो: तिथे खरा धोका (आणि तो प्रश्नच बहुधा यंत्राच्या “धूसर-दाब” प्रदेशातला आहे: 2.4: सावध व्हा). मोजायचं ते हे: शब्द नाही.

MAP वर

या chapter ची चौकट थेट करार-भाषेत गेली आहे: enterprise- ग्राहक आता “reproducibility” विचारतात (तेच input, तेच output मिळेल का?): audit-कारणांसाठी (bank ने ग्राहक का नाकारला हे **पुन्हा दाखवता** आलं पाहिजे: Book 1, 6.4 च्या रोजकीर्दीची मागणी, AI वर). विक्रेते देतात: temperature-ताबा, seed (फाशांचा ठरलेला डाव), आणि प्रत्येक उत्तराची नोंद (कुठलं model-रूप, कुठली रांग). तुमच्या product मध्येही हीच शिस्त स्वस्तात येते: **AI-निर्णयाची नोंद ठेवा: रांग + उत्तर + model-नाव**: उद्याचा वाद आजच्या एका log-ओळीने मिटतो.

स्वतः बघा (5 मिनिटं)

गाभा-स्थिरतेची चाचणी: एकच प्रश्न **तीन नव्या chats** मध्ये (रांग-फरक टाळायला) विचारा: आधी सर्जनशील (“दुकानासाठी घोषवाक्य सुचव”): तिन्ही उत्तरं वेगळी येतील: **हवीच** होती. मग नियम-छाप (“28 फेब्रुवारी 2025 ला आठवडा कुठला?”): तिन्ही सारखी यायला हवीत. आता तुमच्याकडे दोन-रकानी नजर आहे: कुठल्या प्रश्नावर विविधता शोभते, कुठल्यावर शंका उठवते: हीच नजर AI-वापराची अर्धी परिपक्वता आहे.

विचार करा

- (derivation) “गाभा दहादा तोच येतो का” ही तपासणी **स्वतःच एक हत्यार** बनवता येते: कसं? (Hint: 1.3-1.4 ची probability- भाषा: अनेक फेऱ्या = काय मोजता येतं?)

उत्तर: तोच प्रश्न 5-10 वेळा फिरवा (फासे उघडे ठेवून!) आणि उत्तरांची **मतमोजणी** करा: 10 पैकी 9 वेळा “refund-पात्र” आलं: गाभा घट्ट, विश्वास ठेवा; 6-4 फाटलं: यंत्र स्वतःच डळमळतंय: हा प्रश्न माणसाकडे पाठवा. म्हणजे विविधता (जी “दोष” वाटत होती) हीच **आत्मविश्वास-मापक** बनली: फुकटात calibration (1.4)! या युक्तीचं बाजारी नाव self-consistency: आणि महाग-महत्त्वाच्या निर्णयांत ती standard होते आहे: दहा फेऱ्यांचा token-खर्च एका चुकीच्या refund पेक्षा स्वस्त असतो. Book 1, 4.2 चं सूत्र शेवटचं: तपासणीचा खर्च चुकीच्या किमतीशी तोला: AI ने तपासणी स्वस्त केली: वापरा.

Chapter 4.5 [SPINE]: हे विसरतं का

लांब संवादाच्या शेवटी तुम्ही म्हणता “सुरुवातीला ठरलेल्या budget-प्रमाणे कर”: आणि यंत्र विचारतं “कुठलं budget?” राग येतो: पण Part 2 च्या शेवटच्या chapter ला आता तुमच्याकडे रागा- ऐवजी यंत्रणा आहे. “विसरणं” फोडून तीन वेगळी दुखणी निघतात:

दुखणं 1: खिडकी संपली. आठवा 2.10: यंत्राची “आठवण” वजनांत नाही, **रांगेत** आहे: आणि रांगेला मर्यादा आहे (context window: 1.7). का मर्यादा? दोन जुने राक्षस: attention चा n-squared (3.4) आणि KV-cache ची memory-भूक (4.2). संवाद खिडकीबाहेर वाढला की apps काय करतात: **रांगेचं डोकं कापतात** (जुनं गाळून पुढे) किंवा जुन्याचा **सारांश** करून ठेवतात. तुमचं budget डोक्यासोबत कापलं गेलं: यंत्र “विसरलं” नाही: त्याच्या जगातून ते **वजा** झालं. शाळेतल्या फळ्याची उपमा नेमकी आहे: शिक्षक लिहीत जातो; फळा भरला की **जुनं पुसूनच** नवं येतं: आणि पुसलेलं शिक्षकाच्या डोक्यात असतं, फळ्याच्या नाही: पण इथे फळाच सगळं आहे: शिक्षक नाहीच!

दुखणं 2: खिडकी मोठी, पण नजर विरळ. आजच्या जाहिराती: “1-2 million tokens ची खिडकी!” (2.10: पुस्तकंच्या पुस्तकं मावतात). मग प्रश्नच मिटला? नाही: मोजणीत सापडलं की **खूप लांब रांगेत यंत्राची जुळणी मधल्या भागावर विरळ** होते: सुरुवात-शेवट घट्ट, मध्य धूसर (3.7 च्या MAP मधलं “lost in the middle”: आता पूर्ण). कारण समजण्याजोगं आहे: attention चे हिस्से (3.5) लाखो पाट्यांवर वाटले की प्रत्येकीला चिमूटभर: महत्त्वाची पाटी गर्दीत हरवते. खिडकी “मावणं” वेगळं, “नीट न्याहाळणं” वेगळं: गोदामात माल मावला म्हणजे सापडेल असं नाही (Book 1, 6.2 ची जुनी जाणीव!).

दुखणं 3: नवं सत्रच. नवा chat = कोरी रांग: काल ठरलेलं आज “नाहीच.” हे विसरणं नाही; **जन्मच नवा** आहे (4.1: file शी बोलताय; ती तुम्हाला ओळखत नाही). Apps वर “memory” नावाची सोय दिसते ती काय: तुमच्याबद्दलच्या नोंदी एका **वहीत** (database: Book 1!) ठेवून **प्रत्येक नव्या रांगेच्या सुरुवातीला चिकटवणं**: आठवणीचा भास, रचनेने बांधलेला. (आणि हो: cache-योग्य जागी: 4.2!)

तिन्ही दुखण्यांवर एकच घरगुती इलाज-चौकट, कामाला लागणारी:

(अ) महत्त्वाचं पुन्हा सांगा. लांब संवादात निर्णय-क्षणी गाभा **परत** द्या (“आठवण: budget 2,000, मराठी-प्रथम”). माणसा- लाही meeting-शेवटी “ठरलं ते पुन्हा” वाचतातच: minutes!

(ब) सारांश-टप्पे. मोठ्या कामात मधून-मधून “आत्तापर्यंत ठरलेलं 5 ओळींत लिही” मागा: आणि **तो सारांश** पुढच्या रांगेचं डोकं बनवा: फळा पुसण्याआधी महत्त्वाचं कोपऱ्यात लिहिणारा हुशार शिक्षक.

(क) वही बाहेर ठेवा. कायमच्या गोष्टी (नियम, ग्राहक-नोंदी) रांगेवर सोपवू नका: database त ठेवा आणि लागेल तेव्हा रांगेत आणा: हीच ती Book 1 + Book 2 ची जोड (2.4 च्या शेवटी दिलेलं वचन), आणि हिचं पूर्ण शास्त्र (शोधून-आणून-देणं: RAG) Part 3 उघडतो.

इथे लोक काय चुकीचं समजतात

“विसरतं म्हणजे कमअस्सल आहे.” उलट चष्मा लावा: **रांग-आठवण ही सोयही आहे**: chat बंद केला की “सगळं विसरलं” याचा अर्थ तुमचं बोलणं वजनांत मुरत नाही (2.10: गोठलेली file!): खासगीपणाची हीच पहिली भिंत. आणि दुसरी गल्लत: “मोठी खिडकी घेतली की प्रश्न मिटला”: आता माहीत आहे: खिडकी **महाग** आहे (token-भाडं + KV-memory: प्रत्येक request ला!), आणि मध्य-धूसरपणा उरतोच. लाख-token खिडकीत सगळं कोंबणं हा आळस आहे; **नेमकं** देणं ही कला: कमी tokens = कमी bill = घट्ट नजर: तिहेरी फायदा.

MAP वर

“आठवण” ही आता AI-products ची **मुख्य लढाई** आहे: ChatGPT/ Claude च्या memory-सोयी, “तुमच्या company चं सगळं जाणणारे” enterprise-assistants, personal-AI ची स्वप्न: सगळ्यांच्या मुळाशी हाच प्रश्न: रांग क्षणिक आहे; टिकाऊ आठवण **बाहेर** बांधावी लागते: आणि जो ती बांधतो, त्याच्याकडे ग्राहकाचा इतिहास = खिळे (Book 1, 8.5 चा switching-cost!) = platform- ताकद. Founder-नजर: तुमच्या product ची “आठवण-वही” (ग्राहकाच्या पसंती, इतिहास, संदर्भ) हीच तुमची खरी मालमत्ता: model कोणाचंही असो: वही **तुमची** (5.1 चा नियम, या part चा शेवटचा मुक्काम).

स्वतः बघा (5 मिनिटं)

खिडकी-कापणी प्रत्यक्ष: एका chat च्या **पहिल्या** message मध्ये एक खूण पेरा (“लक्षात ठेव: माझा आवडता रंग जांभळा”). मग लांबलचक गप्पा मारा (मोठे मजकूर paste करा: रांग फुगवा). अधून-मधून विचारा: “माझा आवडता रंग?” सुरुवातीला बिनचूक: खूप लांबल्यावर एक क्षण येईल: चाचपडणं किंवा प्रामाणिक “तुम्ही सांगितलं नाही”: **तो क्षण म्हणजे तुमची खूण खिडकीबाहेर पडल्याचा**. आता (अ)-इलाज लावा: खूण पुन्हा सांगा: लगेच रुळावर. विसरण्याची यंत्रणा आणि इलाज: दोन्ही, एका प्रयोगात.

विचार करा

- (derivation) Part 2 चा शेवट-प्रश्न, पुढच्या part चं दार: “सगळं रांगेत कोंबणं” महाग-धूसर, आणि “वजनांत घालणं” अशक्य-गोठलेलं: मग तिसरा रस्ता कुठला असेल? तुमच्या टिफिन-app ला 2,000 ग्राहकांच्या नोंदी यंत्राला “माहीत” करून द्यायच्या आहेत: रचना सुचवा.

उत्तर: तिसरा रस्ता: गरजेपुरतं, शोधून, रांगेत. नोंदी database त (Book 1, 6.1); ग्राहकाचा प्रश्न आला की त्याच्या- पुरत्या नोंदी शोधा: आणि शोध शब्दांनी नाही, अर्थाने (3.1 चं embedding: “डबा उशिरा” आणि “delivery late” एकच सापडावेत!): मिळालेल्या 5-10 नोंदी प्रश्नासोबत रांगेत चिकटवा: यंत्र “जाणणाऱ्या”सारखं उत्तर देतं: खिडकी छोटी, नजर घट्ट, bill हलकं. ही रचना (शोधा-आणा-द्या: Retrieval + Generation = **RAG**) आजच्या जवळजवळ प्रत्येक “आमचा data जाणणारा AI” product चा सांगाडा आहे: आणि Part 3 च्या पहिल्याच दारात ती पूर्ण भेटेल. Part 2 इथे संपतो: यंत्र आतून पूर्ण उघडलं; आता त्याला **कामाला** लावायचं.

BOOK 2, PART 2 चा शेवट: नकाशा आणि पुढचं पाऊल

जे तुमच्याकडे आता आहे

EMBEDDING	शब्द -> अर्थ-नकाशावरची जागा (vector); जवळीक = संगत; दिशा = नाती (राजा->राणी = काका->काकी)
संदर्भनि अर्थ	"पान" ची जागा वाक्यागणिक सरकते (contextual); अर्थ शब्दात नाही, वाक्यात
ATTENTION	प्रत्येक शब्दाचा बाजार: मागणी x पाटी = जुळणी, हिश्यांनी मालाची मिसळण (query-key-value)
MULTI-HEAD	एकाच क्षणी अनेक स्वतंत्र नजरा; भूमिका उगवतात, वाटल्या जात नाहीत
क्रम-शिकका	बाजाराला अंतर दिसत नाही: जागेचा गंध token-vector मध्ये (positional; क्रम रचनेतून data त)
TRANSFORMER	मजला = बाजार + एकांत; मजले x 30-100; खालून वर: शब्द -> वाक्य -> हेतू; GPT-Claude-Gemini सगळ्यांचा एकच सांगाडा (2017)
SCALING	मोठं करा -> चूक नियमित घटते (सरळ रेषा) + क्षमतांच्या उड्या (emergence); आता फाटे: विचार-वेळ (thinking), चांगला data
MULTIMODAL	photo=चौकोन, आवाज=काळ-तुकडे, video+=क्षण: कापणी वेगळी, इमारत एक; चष्मा=मर्यादा
कारखाना	model=भरलेली तोफ (GPU-memory त); पंगत-दळण (batching); token-भाव = GPU-भाड्याचं किरकोळ रूप
PREFILL/DECODE	वाचन झपाट्याचं (समांतर), जन्म थेंबाचा (क्रमिक); KV-cache = पाट्या पुन्हा न मोजणं (10% ची सूट!)
वेगाचे आकडे	TTFT (पहिला शब्द) vs लय (tokens/sec) vs लांबी; streaming = वाट दिसते
वेगळं उत्तर	फासे (temperature) + वेगळी रांग + धूळ; मोजा गाभा-स्थिरता; मतमोजणी = फुकट confidence
विसरणं	आठवण रांगेत; खिडकी कापली जाते; मोठी खिडकी = महाग + मध्य-धूसर; इलाज: पुन्हा सांगा, सारांश, वही बाहेर (RAG चं दार)

एक परीक्षा, स्वतःसाठी

पुस्तक न उघडता, बोलून:

1. "राजा - पुरुष + स्त्री = राणी" हे यंत्रात कोणी घातलं?
2. Attention ने जुन्या पद्धतीची कुठली दोन दुखणी मारली?
3. "आमची खिडकी 2 million tokens" ऐकल्यावर कुठले दोन प्रश्न विचाराल?
4. API-bill मध्ये cache-hit ला 10% किंमत का पुरते?

अडलात तर: 1 -> 3.2, 2 -> 3.4, 3 -> 4.5, 4 -> 4.2.

Part 3 मध्ये काय आहे

यंत्र आतून पूर्ण उघडलं. पण दोन मोठे प्रश्न मुद्दाम शिल्लक आहेत: **(1)** हे इतकं हुशार यंत्र ठाम आत्मविश्वासाने **खोटं** का बोलतं: आणि त्याचं उत्तर कसं तपासायचं? **(2)** बोलणाऱ्या यंत्राला **हात** कसे दिले जातात: tools, agents, आणि तुमचं रोजचं Claude Code आतून कसं चालतं? आणि शेवटी, ज्यासाठी हा सगळा प्रवास होता: **AI च्या जगात पैसा नेमका कोण कमावतो, आणि एकटा माणूस 2026 मध्ये काय बांधू शकतो.** Part 3: चुका, हात आणि पैसा: दोन्ही पुस्तकांचा शेवटचा आणि सगळ्यात व्यापारी part: अंतिम परीक्षेसह.

PART 2 चॢ MINI-GLOSSARY

attention	मागणी-पाटी जुळणीने वजनदार मिसळण (3.5)
batching	अनेकांचे प्रश्न एका GPU-फेरीत दळणं (4.1)
contextual embedding	वाक्यागणिक सरकणारी अर्थ-जागा (3.3)
decode	जन्म-टप्पा: token-by-token, क्रमिक (4.2)
embedding/vector	शब्दाची अर्थ-नकाशावरची जागा (3.1)
emergence	आकाराच्या उंबरठ्यावर उगवणाऱ्या क्षमता (3.9)
head/multi-head	एक नजर / समांतर अनेक नजरा (3.6)
KV-cache	मोजलेल्या पाट्या-मालाची साठवण (4.2)
layer (मजला)	बाजार + एकांत; इमारतीची वीट (3.8)
lost in middle	लांब रांगेत मध्य-धूसर नजर (4.5)
positional encoding	क्रमाचा शिक्का token-vector मध्ये (3.7)
prefill	वाचन-टप्पा: पूर्ण prompt समांतर (4.2)
query/key/value	मागणी / पाटी / माल: बाजाराचे तीन कागद (3.5)
RAG (दार)	शोधा-आणा-रांगेत-द्या ची रचना (4.5)
reasoning/thinking	उत्तराआधी विचार-tokens खर्चणं (3.9)
scaling laws	मोठेपणा -> चूक-घट ची नियमित रेषा (3.9)
seed	फाशांचा ठरवलेला डाव (4.4)
self-consistency	अनेक फेऱ्यांची मतमोजणी = विश्वास-मापक (4.4)
streaming	थेब दिसू देणं; वाट सुसह्य (4.3)
transformer	attention-मजल्यांची इमारत; 2017 (3.8)
TTFT	पहिल्या शब्दाची वाट; voice मध्ये राजा (4.3)